



Research Intake 2026

Day 9

Advanced Deep Learning & Transfer Learning

A Beginner's Guide for Students

Understanding AI, Machine Learning, and Deep Learning

Gudsky Research Foundation

Table of Contents

01 — Transfer Learning Fundamentals

- 1.1 What Is Transfer Learning?
- 1.2 Feature Extraction vs Fine-Tuning vs Full Retraining
- 1.3 When to Use Transfer Learning — Decision Framework
- 1.4 Domain Adaptation

02 — Pretrained Models & Backbones

- 2.1 ImageNet & Why Pretrained Weights Matter
- 2.2 Popular Backbone Families — Deep Theory
- 2.3 Vision Transformers (ViT) — A Paradigm Shift
- 2.4 Choosing the Right Backbone

03 — Fine-Tuning Strategies

- 3.1 Layer Freezing — The Theory
- 3.2 Progressive Unfreezing — Preventing Catastrophic Forgetting
- 3.3 Discriminative Learning Rates — The Intuition
- 3.4 The Complete Fine-Tuning Mental Model

04 — Advanced Data Augmentation

- 4.1 Why Standard Augmentation Has Limits
- 4.2 Mixup — Teaching Linearity Between Classes
- 4.3 CutMix — Teaching Part-Based Recognition
- 4.4 RandAugment & AutoAugment
- 4.5 Test-Time Augmentation (TTA)
- 4.6 Augmentation Technique Comparison

05 — Attention Mechanisms & Transformers

- 5.1 The Problem Attention Was Designed to Solve
- 5.2 Self-Attention — Query, Key, Value
- 5.3 Multi-Head Attention — Multiple Perspectives
- 5.4 The Transformer Architecture — Layer by Layer
- 5.5 BERT vs GPT — A Deep Conceptual Comparison
- 5.6 Vision Transformer (ViT) — Attention for Images

06 — Advanced Training Techniques

- 6.1 Gradient Clipping
- 6.2 Label Smoothing
- 6.3 Mixed Precision Training
- 6.4 Gradient Accumulation
- 6.5 Exponential Moving Average (EMA) of Weights

07 — Model Compression & Deployment

- 7.1 The Production Trilemma
- 7.2 Pruning — Removing What Doesn't Matter
- 7.3 Quantization
- 7.4 Knowledge Distillation
- 7.5 ONNX — The Universal Deployment Format

08 — Generative Models Overview

- 8.1 GANs — The Adversarial Game
- 8.2 Variational Autoencoders — Structured Latent Space
- 8.3 Diffusion Models — The Denoising Revolution
- 8.4 GAN vs VAE vs Diffusion — Comparison

09 — Hands-On Labs

10 — Glossary & Review Questions

Learning Objectives

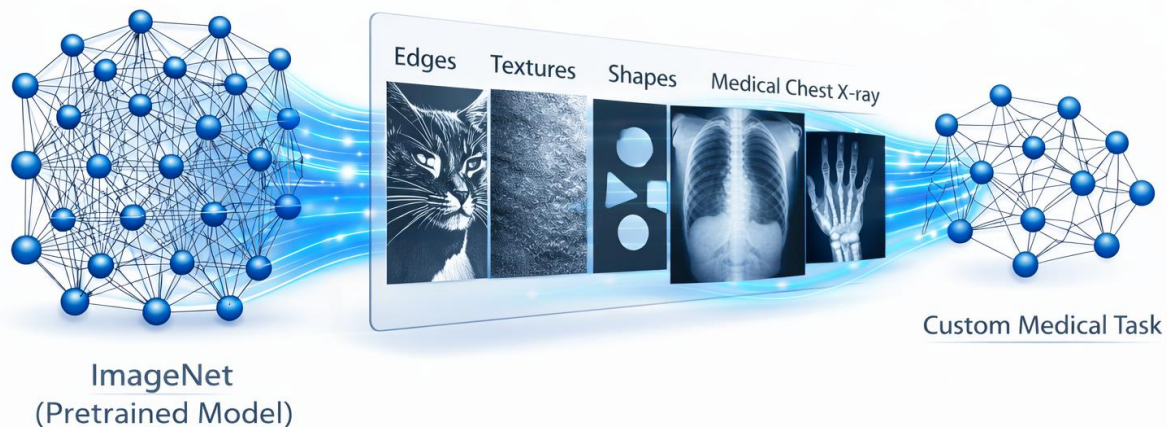
By the end of this session, students will be able to:

1. **Explain** what transfer learning is and why pretrained weights dramatically accelerate training on new tasks.
2. **Distinguish** between feature extraction, fine-tuning, and full retraining — and select the right strategy for any situation.
3. **Apply** layer freezing and progressive unfreezing to prevent catastrophic forgetting.
4. **Implement** discriminative learning rates across layer groups for stable fine-tuning.
5. **Apply** advanced data augmentation: Mixup, CutMix, and RandAugment.
6. **Explain** self-attention using the Query, Key, Value formulation and trace the scaled dot-product computation.
7. **Describe** the full Transformer encoder-decoder architecture layer by layer.
8. **Distinguish** BERT (bidirectional encoder) from GPT (causal decoder) and explain when to use each.
9. **Explain** mixed precision training and gradient accumulation and why they matter for large models.
10. **Describe** knowledge distillation and the teacher-student training pipeline.
11. **Understand** how trained models are exported for deployment using ONNX.

01 — Transfer Learning Fundamentals

Day 9 begins with one of the most practically important ideas in modern deep learning: instead of training a model from scratch on every new task, we can start from a model already trained on a large dataset — and adapt it. This idea, called transfer learning, has transformed how AI systems are built in the real world.

Transfer Learning



1.1 What Is Transfer Learning?

Transfer learning is the practice of taking a model trained on one task — called the source task — and reusing it as a starting point for a different but related task, called the target task. Instead of initialising a network's weights randomly and learning everything from scratch, you begin with weights that have already learned powerful representations from vast amounts of data.

To understand why this works, consider how neural networks learn. In a deep network trained on images, the early layers learn to detect simple patterns: edges, corners, gradients, and basic textures. These patterns appear in virtually every image, whether it is a photograph of a cat, an X-ray of a lung, or a satellite image of farmland. The middle layers combine these patterns into more complex structures — curves, grids, shapes, and object parts. Only the deepest layers, closest to the output, learn features that are specific to the original task.

This layered structure means that most of the network's knowledge is general and reusable. By preserving these layers and only adapting the task-specific final layers, a model trained on millions of images can be repurposed for an entirely new problem using only a small fraction of the data that would otherwise be required.

💡 Why Transfer Learning Works

Early layers detect edges, corners, and textures — universal features found in all images.
Middle layers detect shapes, object parts, and patterns — broadly applicable across domains.
Only final layers learn task-specific features. Everything before them is reusable knowledge.
A model trained on 1.2 million ImageNet images has already solved the hard part of visual recognition.

You only need to adapt the final layers — which requires far less data and compute.

A helpful analogy: learning to drive a car is much easier if you already know how to ride a bicycle. Balance, spatial awareness, and road sense all transfer. You only need to learn the new controls — the steering wheel, pedals, and mirrors. Transfer learning works on exactly the same principle.

Real-World Example: Medical Imaging AI

In 2016, researchers at Google trained a neural network to detect diabetic retinopathy — a serious eye disease — from retinal photographs.

Rather than training from scratch on medical images (which are expensive to label), they started with a ResNet pretrained on ImageNet natural photos.

The model had never seen a retinal image, yet its pretrained knowledge of edges, textures, and shapes gave it a powerful starting point.

After fine-tuning on a relatively modest dataset of labelled retinal images, it achieved performance matching expert ophthalmologists.

This would have been impossible without transfer learning — the medical dataset alone was too small to train a deep network from scratch.

1.2 Feature Extraction vs Fine-Tuning vs Full Retraining

There are three main strategies for applying a pretrained model to a new task. The right choice depends on how much labelled data you have and how similar your new task is to the original training task.

Feature Extraction is the most conservative strategy. The entire pretrained backbone is frozen — its weights never change — and only a new classification head added on top is trained. The pretrained network acts as a fixed feature extractor: it converts raw images into rich numerical representations, and the new head learns to classify those representations. This works remarkably well when the new dataset is small and the new task is similar to the source task.

Fine-Tuning unfreezes some or all of the pretrained layers and continues training them alongside the new head. Because the pretrained weights are already close to a good solution, fine-tuning uses a much smaller learning rate than training from scratch. This preserves the valuable knowledge already encoded in the weights while adapting the representations to the new domain.

Full Retraining initialises the network randomly and trains every layer from scratch on the new dataset. This is only appropriate when the new domain is radically different from the source domain and when a very large labelled dataset is available. Even in this case, using pretrained weights as a starting point often helps — random initialisation is rarely the best choice.

Strategy	Frozen Layers	Trainable Layers	Data Needed	Best When
Feature Extraction	Entire backbone	New head only	Very small (<1K samples)	Task similar to source; tiny dataset
Partial Fine-Tuning	Early layers	Last few layers + head	Small–Medium (1K–10K)	Moderate similarity;

Strategy	Frozen Layers	Trainable Layers	Data Needed	Best When
				limited GPU memory
Full Fine-Tuning	None	All layers + head	Medium–Large (>10K)	Sufficient data; maximum accuracy needed
Full Retraining	N/A (random init)	All layers from scratch	Very Large (>100K)	Target domain very different from source



Real-World Example: Two Very Different Projects

A startup building a plant disease detector had only 500 labelled photos of diseased leaves.

They used feature extraction: froze all of ResNet-50, trained only a small classification head. Result: 87% accuracy with almost no training cost.

A hospital with 50,000 chest X-rays labelled by radiologists used full fine-tuning on the same base model. By retraining all layers with a careful learning rate schedule, they achieved 94% accuracy — matching the clinical benchmark.

Same base model, same approach, different strategies — chosen purely based on dataset size.

1.3 When to Use Transfer Learning — Decision Framework

The choice of strategy depends on two key factors: how much labelled data you have, and how similar your domain is to the source domain (typically ImageNet for vision tasks). These two dimensions create a four-quadrant decision framework.

Quadrant	Dataset Size	Domain Similarity	Recommended Strategy
Q1	Small (<1K)	Similar (e.g., flowers, pets)	Feature extraction only — freeze backbone, train head
Q2	Small (<1K)	Different (e.g., X-rays, satellite)	Fine-tune upper layers carefully — small LR, early stopping
Q3	Large (>10K)	Similar	Fine-tune all layers with discriminative LR — fast convergence
Q4	Large (>10K)	Different	Full fine-tuning or retrain — pretrained init still helps



Real-World Examples by Quadrant

Q1 — Dog breed classifier with 800 labelled photos. ImageNet already knows dogs well. Feature extraction achieves excellent results.

Q2 — Satellite crack detection in concrete with 300 images. Domain is very different from natural photos. Carefully fine-tune only the upper layers.

- Q3 — 50,000 product photos for e-commerce categorisation. Similar to ImageNet; large dataset. Fine-tune all layers aggressively.
- Q4 — Industrial thermal imaging with 200,000 samples. Thermal cameras capture heat, not colour. Full fine-tuning is needed; but pretrained initialisation still beats random weights.



Practical Rule of Thumb

- When in doubt, always start with feature extraction (freeze all, train head only).
- If validation accuracy plateaus, gradually unfreeze more layers.
- Always use a smaller learning rate for pretrained layers than for the new head.
- Never use the same learning rate for both pretrained and new layers — it can destroy the pretrained knowledge.

1.4 Domain Adaptation — Bridging Distribution Gaps

Domain adaptation addresses a subtler challenge: even when the task is the same (e.g., image classification), performance can drop dramatically if the statistical characteristics of the data differ between training and deployment. This mismatch is called domain shift or distribution shift.

Domain shift arises in many real-world settings. A model trained on daytime driving footage may fail at night. A sentiment classifier trained on movie reviews may struggle with product reviews. A medical model trained in one hospital may underperform at another due to differences in imaging equipment or patient demographics.

Domain Shift Type	Example	Solution
Covariate Shift	Training: natural photos Target: medical X-rays. Same task, different visual statistics.	Fine-tune all layers with a small learning rate to shift the feature distribution gradually.
Label Shift	Source has 1000 classes Target has 5 classes. Class balance differs radically.	Replace output layer, retrain the head, optionally fine-tune upper backbone layers.
Dataset Bias	Source: high-res studio photos Target: low-res phone camera images.	Add noise, blur, and resolution augmentation to bridge the quality gap.
Temporal Shift	Model trained on 2020 data, deployed in 2025. World has changed.	Periodic retraining on fresh data; monitor performance metrics in production.



Real-World Example: Tesla Autopilot and Geographic Domain Shift

- Tesla's Autopilot was primarily trained on driving data from sunny California and similar climates. When deployed in Finland during winter, the system encountered snow-covered roads, white road markings hidden under snow, and dramatically different lighting conditions.

This is a classic case of covariate shift: the task is the same (navigate safely), but the visual distribution of the input data is completely different.

Tesla's response was continuous fleet learning — collecting edge-case data from real vehicles globally and periodically retraining to reduce distribution gaps.

This story illustrates why domain adaptation is not just an academic concern — it is a safety-critical engineering challenge.

02 — Pretrained Models & Backbones

A backbone is the main feature-extraction network — typically a CNN or Transformer — pretrained on a large dataset like ImageNet. Choosing the right backbone determines your model's accuracy, speed, and memory footprint. This section surveys the most important backbone families and builds intuition for when to choose each.

2.1 ImageNet & Why Pretrained Weights Matter

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) transformed deep learning. The dataset contains 1.2 million training images across 1,000 categories — diverse enough that the representations learned from it generalise broadly across visual tasks. Every major advance in computer vision from 2012 onward was first demonstrated on ImageNet.

The reason ImageNet pretraining transfers so well is the hierarchical nature of what the model learns. A model does not simply memorise cats and dogs — it learns a universal visual vocabulary that can be applied to almost any image-based task.

What a Model Trained on ImageNet Already Knows

Layers 1–2: Gabor-like edge detectors, colour blobs, simple textures — completely universal across all images.

Layers 3–5: Corners, curves, repeating textures, grids, honeycombs — broadly applicable to any visual domain.

Layers 6–8: Object parts — eyes, wheels, faces, leaves — already shaped like real-world visual components.

Layers 9+: Full objects and class-specific features — these are the layers you replace for your task.

Key insight: Only the last 1–2 layers are task-specific. Everything before them is reusable knowledge.

Real-World Example: Andrej Karpathy's Feature Visualisations

Andrej Karpathy (formerly AI Director at Tesla) published famous visualisations showing what individual neurons in a deep network respond to.

A neuron in layer 2 consistently lit up for diagonal edges, regardless of whether it was looking at a cat, a car, or a landscape.

A neuron in layer 6 consistently responded to dog faces — curved ears, wet noses, round eyes — whether the dog was a poodle or a Labrador.

A neuron in layer 8 responded to text: newspaper headlines, book covers, street signs — a concept never explicitly taught.

These emergent, reusable features are exactly why transfer learning works so reliably across domains.

2.2 Popular Backbone Families — Deep Theory

Different backbone architectures reflect different design philosophies. Understanding the ideas behind each family helps you choose the right tool for your task.

ResNet — The Skip Connection Revolution

Before ResNet (He et al., 2015), training very deep networks was nearly impossible due to the vanishing gradient problem. As gradients flow backward through many layers during training, they become exponentially smaller — eventually becoming so tiny that early layers receive essentially no learning signal and stop improving.

ResNet's insight was elegantly simple: add a skip connection that bypasses each block and adds the input directly to the output. This creates a gradient highway — even if the gradients through the convolutional layers are weak, the skip connection ensures that a strong gradient signal can always flow backward to early layers. Networks with hundreds of layers suddenly became trainable.

ResNet also introduced the concept of residual learning — instead of asking each block to learn a complete mapping, it only needs to learn the residual, or difference, from the identity. Learning a small correction is much easier than learning a full transformation.

EfficientNet — Compound Scaling

EfficientNet (Tan & Le, Google Brain, 2019) challenged the conventional wisdom that making a network deeper, wider, or higher-resolution independently improves accuracy. They showed that these three dimensions interact: making a network deeper without making it wider leaves capacity on the table; increasing resolution without proportionally increasing depth leaves the network unable to capture fine-grained features.

Their compound scaling rule uses a grid search to find the optimal ratio between depth, width, and resolution — then scales all three simultaneously. EfficientNet-B0 was the baseline model, and B1 through B7 applied compound scaling at increasing magnitudes. The result was a family of models that achieved state-of-the-art accuracy with dramatically fewer parameters than prior approaches.

MobileNet — Efficiency Through Factorisation

MobileNet (Howard et al., Google, 2017) was designed for a specific constraint: running on mobile phones and embedded devices with limited compute and memory. The key innovation was depthwise separable convolutions, which factorise a standard convolution into two sequential operations.

A standard convolution applies K filters to all input channels simultaneously. A depthwise separable convolution first applies K spatial filters — one per input channel — then applies 1×1 pointwise convolutions to combine the results. The mathematical result is nearly identical, but the computational cost is approximately 8–9 times lower. This made deep learning practical on hardware with no GPU.

Backbone	Parameters	Top-1 Accuracy	Inference Speed	Best Use Case
ResNet-50	25M	76.1%	Fast	General baseline; excellent transfer learning anchor
ResNet-101	44M	77.4%	Medium	Higher accuracy when compute allows
EfficientNet-B0	5.3M	77.1%	Very Fast	Mobile/edge deployment; excellent accuracy per parameter
EfficientNet-B4	19M	82.9%	Medium	Production: balanced accuracy and speed
EfficientNet-B7	66M	84.3%	Slow	Maximum accuracy; competition and research settings

Backbone	Parameters	Top-1 Accuracy	Inference Speed	Best Use Case
MobileNetV3-Small	2.5M	67.7%	Extremely Fast	On-device inference; real-time mobile apps
ViT-Base/16	86M	81.8%	Medium	Large datasets; state-of-the-art understanding tasks
ViT-Large/16	307M	82.6%	Slow	Research; requires very large datasets to shine

🌐 Real-World Backbone Usage in Industry

Instagram uses EfficientNet for real-time content moderation — classifying billions of images daily with tight latency requirements.

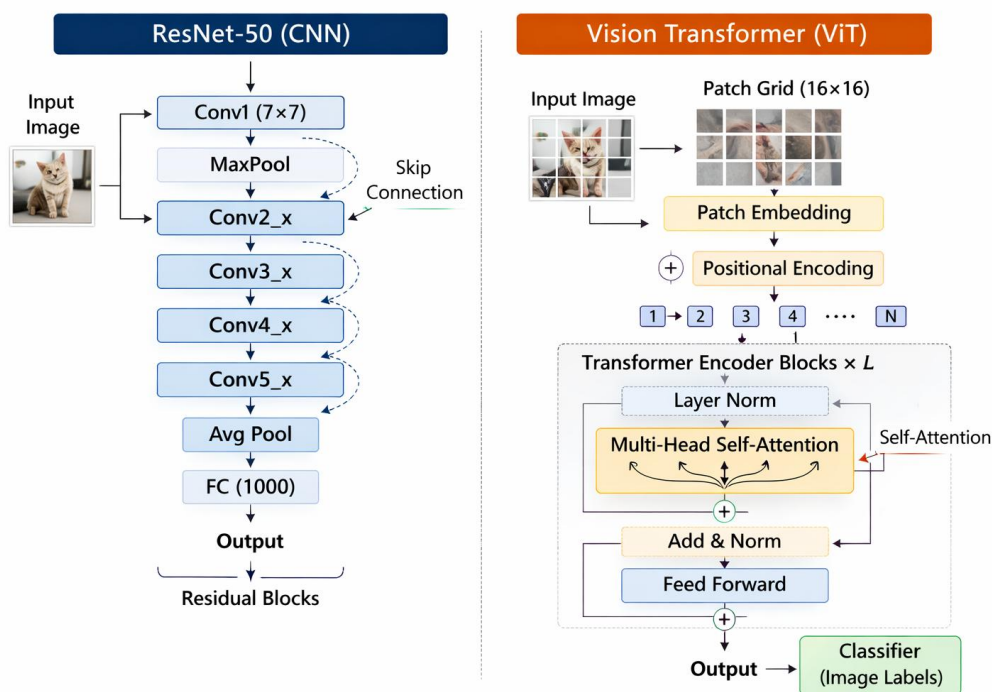
YouTube uses ResNet variants for video understanding — predicting categories, detecting inappropriate content, and generating thumbnails.

Apple's Face ID chip runs a heavily compressed MobileNet variant — achieving sub-second recognition entirely on-device with no internet connection.

Google's Lens uses ViT-based models for scene understanding — the global attention mechanism helps it understand context across the full image.

2.3 Vision Transformers (ViT) — A Paradigm Shift

Vision Transformer (ViT, Dosovitskiy et al., 2020) was a radical departure from decades of CNN dominance. The paper's central claim was bold: a pure Transformer applied directly to sequences of image patches can match or exceed CNN performance — with no convolutions at all.



CNNs have a built-in inductive bias: they assume that nearby pixels are more related than distant ones (locality), and that the same feature detector applies everywhere in the image (translation equivariance). These assumptions are powerful when data is limited — they constrain the model's search space helpfully.

Transformers have no such biases. Every patch can attend to every other patch equally. This means the model must learn spatial relationships from data — which requires more data — but also means it can discover long-range relationships that CNNs structurally cannot. A CNN processing a face always looks at local patches; a ViT can simultaneously relate the eye to the ear to the chin to contextual background elements.

 How ViT Processes an Image — Step by Step

Step 1 — Patch Splitting: Divide the 224×224 image into 16×16 patches. This produces 196 patches (tokens).

Step 2 — Linear Projection: Each patch is flattened to 256 values and projected to a 768-dimensional embedding.

Step 3 — Positional Encoding: Add learned position embeddings so the model knows where each patch came from.

Step 4 — [CLS] Token: A special classification token is prepended — its final state summarises the whole image.

Step 5 — Transformer Encoder: Standard multi-head self-attention + feed-forward blocks process all 197 tokens.

Step 6 — Classification Head: A linear layer on the [CLS] token output produces the final class probabilities.

Criterion	CNN (ResNet)	Vision Transformer (ViT)	Winner
Small datasets (<10K)	Excellent — inductive bias helps	Poor — needs massive data to learn spatial structure	CNN
Large datasets (>100K)	Very good	Excellent — attention scales better with data	ViT
Long-range relationships	Limited by receptive field growth	Global from the very first layer	ViT
Compute efficiency	Parameter efficient	Expensive — $O(n^2)$ attention over patches	CNN
Transfer learning	Excellent — well-studied	Excellent with sufficient source data	Tie
Current state of the art	Strong (ConvNeXt variants)	Dominant at scale	ViT

2.4 Choosing the Right Backbone

Backbone selection is a practical engineering decision that balances accuracy, speed, memory, and data size. There is no universally best backbone — the right choice depends on your constraints.

Backbone Selection Guide

Mobile / Edge deployment → MobileNetV3-Small or EfficientNet-B0. Priority: size and inference speed.

Balanced production system → ResNet-50 or EfficientNet-B3. Priority: accuracy vs speed tradeoff.

Maximum accuracy (research/competition) → EfficientNet-B7 or ViT-Large. Priority: accuracy above all.

Limited labelled data (<1K samples) → ResNet-50 pretrained. Strong inductive bias helps with small data.

Very large labelled dataset (>100K) → ViT-Base or ViT-Large for best results.

Real-time video processing → MobileNetV3 or EfficientNet-B0 with hardware optimisation.

Real-World Example: How Shopify and DeepMind Chose Backbones

Shopify needed to automatically tag millions of merchant product images daily with tight API latency budgets.

They chose EfficientNet-B3 — not the most accurate, but the best accuracy-per-millisecond tradeoff at their scale.

DeepMind, building a research model to understand complex scientific figures, needed maximum capability without latency constraints.

They used ViT-Large as the backbone — prioritising representational power over deployment efficiency.

Same task category (image understanding), completely different constraints, completely different backbone choices.

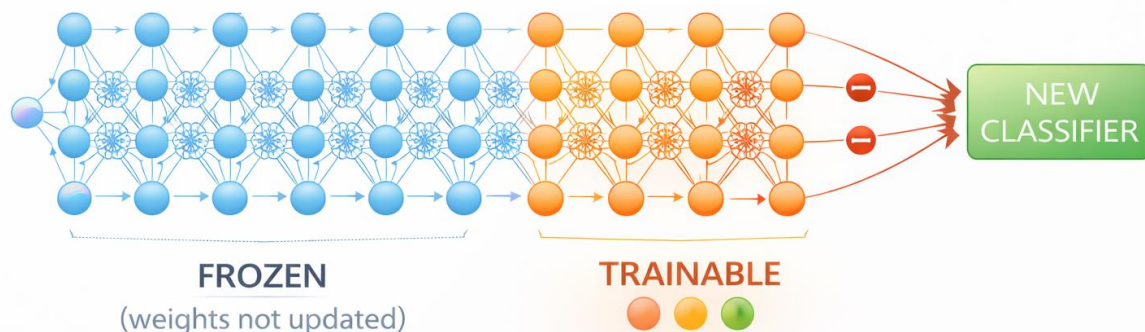
03 — Fine-Tuning Strategies

Fine-tuning is the process of continuing to train a pretrained model on your new dataset. Done correctly, it adapts the model's representations to your domain while preserving the valuable pretrained knowledge. Done incorrectly, it destroys that knowledge — a phenomenon called catastrophic forgetting. This section covers the theory behind the three main fine-tuning strategies.

3.1 Layer Freezing — The Theory

When we freeze a layer, we prevent its weights from receiving gradient updates during backpropagation. The layer still performs its computation during the forward pass — it still transforms activations — but its parameters remain fixed at their pretrained values.

The intuition for which layers to freeze follows from the feature hierarchy: early layers capture universal, low-level features that are useful for almost any visual task. There is rarely a reason to retrain them. Late layers capture task-specific features learned for the original task (e.g., recognising 1,000 ImageNet categories). These must be replaced or retrained for a new task.



A common mistake is to freeze layers by name without considering BatchNorm layers. Standard layer freezing (setting `requires_grad = False`) prevents weight updates but does not freeze BatchNorm's running statistics — its running mean and variance still update based on the new data. When the new data has a different distribution from ImageNet, this creates a silent interference: the pretrained convolutional weights expect certain activation statistics, but the BatchNorm layers are silently shifting them. The solution is to also call `eval()` on frozen BatchNorm modules.

⚠ Critical: The BatchNorm Freezing Problem

Setting `requires_grad = False` on a layer prevents weight updates during backpropagation.

However, BatchNorm layers also maintain running statistics (mean and variance) that update during the forward pass.

If your new data has different statistics from ImageNet, these running stats drift — corrupting the pretrained features.

Solution: Call `module.eval()` on frozen BatchNorm layers. This freezes running statistics while allowing gradient flow.

This is one of the most common silent failures in transfer learning pipelines.

🌐 Real-World Example: OpenAI CLIP as a Frozen Feature Extractor

OpenAI's CLIP model was trained on 400 million image-text pairs from the internet. Its vision encoder produces image embeddings of remarkable quality — capturing semantic meaning, not just visual patterns.

Many downstream applications use CLIP with a completely frozen vision encoder: the encoder is never retrained, only a lightweight head on top is trained.

This works because CLIP's representations are so rich and general that even simple linear classifiers on top of frozen CLIP features outperform fully trained smaller models.

The success of frozen CLIP demonstrates that a sufficiently powerful pretrained feature extractor can make fine-tuning unnecessary.

3.2 Progressive Unfreezing — Preventing Catastrophic Forgetting

Catastrophic forgetting is one of the most important phenomena in deep learning. When a neural network learns a new task, the gradient updates that improve performance on the new task can overwrite the weights that encoded knowledge from the old task. In the extreme case, the model becomes excellent at the new task and completely forgets everything it knew before.

Progressive unfreezing is a technique that mitigates this risk. Instead of unfreezing all layers at once and training the full network, layers are thawed one group at a time — starting from the layers closest to the output and moving backward toward the input. In the first phase, only the new head is trained. In the second phase, the deepest backbone layer group is also unfrozen. This continues phase by phase until all layers are training together.

The logic is that the deepest layers are the most task-specific and need the most adaptation. The earliest layers are the most universal and should change the least. By allowing deeper layers to adapt first, the model's foundational representations remain stable while the higher-level features gradually shift toward the new domain.

Real-World Example: Progressive Unfreezing for NLP Fine-Tuning

Jeremy Howard and Sebastian Ruder introduced progressive unfreezing in their ULMFiT paper (2018) for fine-tuning language models.

They showed that naively fine-tuning all LSTM layers at once on a new text dataset caused catastrophic forgetting of language understanding.

Progressive unfreezing — starting from the final LSTM layer and working backward — preserved the model's general language knowledge while adapting to the new domain.

This technique later became standard practice for fine-tuning BERT and GPT-style models.

The analogy: it is like teaching a new skill to an expert by first adjusting their specialised knowledge, then slowly refining their foundational understanding.

3.3 Discriminative Learning Rates — The Intuition

Even when all layers are unfrozen, they should not all receive the same learning rate. The learning rate controls how much a weight changes in response to a gradient update. A large learning rate means large changes — which is fine for a randomly initialised head that needs to learn from scratch, but catastrophic for carefully tuned pretrained weights that are already close to a good solution.

Discriminative learning rates (also called differential learning rates) assign different learning rates to different layer groups. The canonical approach: early layers receive the smallest learning rate (perhaps

0.001 of the base LR), middle layers receive a moderate rate, and the new head receives the full learning rate. This way, the fundamental visual features change slowly and carefully, while the new task-specific layers adapt quickly.

Discriminative Learning Rate Formula

If the base learning rate is η , then:

Early layers (layers 1–3): $\eta / 100$ — tiny updates to preserve universal features

Middle layers (layers 4–7): $\eta / 10$ — moderate updates to adapt mid-level features

Late layers (layers 8+): η — full updates for task-specific features

New head: $\eta \times 10$ or η — learn from scratch at full or amplified rate

The exact ratios are a hyperparameter, but the principle — smaller LR for earlier layers — is universal.

Real-World Example: BERT Fine-Tuning on Legal Documents

Hugging Face researchers fine-tuning BERT on legal contract analysis found a dramatic sensitivity to learning rate assignment.

Using a uniform learning rate across all transformer blocks caused catastrophic forgetting of grammatical understanding.

Using discriminative rates — 10× smaller for the lower transformer blocks — preserved BERT's language knowledge while adapting to legal vocabulary.

The lower blocks retained their understanding of sentence structure and syntax; the upper blocks specialised in legal concepts.

This became the recommended approach for all BERT fine-tuning in production NLP systems.

3.4 The Complete Fine-Tuning Mental Model

A robust fine-tuning pipeline typically proceeds in two or three phases, combining all three techniques: layer freezing establishes a safe starting point, progressive unfreezing prevents catastrophic forgetting, and discriminative learning rates ensure stable convergence.

Phase	Duration	What Trains	Learning Rate Strategy	Goal
Phase 1: Head Only	5–10 epochs	New classification head only; backbone frozen	Full LR on head	Get head to a reasonable starting point before touching backbone
Phase 2: Progressive Unfreeze	5 epochs per group	Unfreeze one layer group per phase, starting from deepest	Discriminative: small LR for newly unfrozen layers	Gradually adapt backbone without catastrophic forgetting
Phase 3: Full Fine-Tuning	10–20 epochs	All layers; monitor validation loss closely	Discriminative across all groups; cosine LR decay	Maximum accuracy; careful regularisation needed

Common Mistake	Symptom	Fix
Same LR for all layers	Backbone weights destroyed in first few steps; validation loss spikes immediately	Use discriminative LR: 10–100× smaller for pretrained layers
Forgetting BatchNorm freezing	Training loss decreases but validation performance is worse than expected	Call eval() on frozen BatchNorm modules explicitly
Unfreezing all layers at once	Good initial progress followed by sudden performance collapse	Use progressive unfreezing: one layer group per phase
Too large learning rate overall	Loss becomes NaN or oscillates wildly	Reduce LR by 10×; use warmup schedule for first few steps
Too few epochs in Phase 1	Head hasn't converged before backbone unfreezes; unstable gradients	Train head for at least 5 epochs before any unfreezing

04 — Advanced Data Augmentation

Data augmentation artificially increases the diversity of the training set by applying transformations to existing examples. Standard augmentation techniques — random flips, crops, colour jitter — are now table stakes. This section covers advanced techniques that go further: they change not just individual images, but how labels are assigned, forcing models to learn more robust, better-calibrated representations.

4.1 Why Standard Augmentation Has Limits

Standard augmentation techniques teach the model that the same object in slightly different positions, orientations, or lighting conditions should produce the same prediction. This is valuable — it builds spatial invariance and colour robustness. But standard augmentation has a fundamental limitation: every training example is still a single, clean image paired with a single, hard label.

Hard labels (e.g., this image is a cat — probability 1.0, all other classes 0.0) encourage overconfidence. A model trained on hard labels learns to be maximally certain, even at the boundaries between visually similar categories. This hurts calibration — the alignment between a model's confidence and its actual accuracy — and can hurt generalisation.

4.2 Mixup — Teaching Linearity Between Classes

Mixup (Zhang et al., 2018) takes two training images and their labels, and blends them proportionally. The blended image is created by taking a weighted average of the two images pixel-by-pixel, and the blended label is the same weighted average of the two label vectors.

Mixup Formula

Given two images x_1 , x_2 and their labels y_1 , y_2 , and a mixing coefficient λ drawn from a Beta distribution:

Mixed image: $\tilde{x} = \lambda \cdot x_1 + (1-\lambda) \cdot x_2$

Mixed label: $\tilde{y} = \lambda \cdot y_1 + (1-\lambda) \cdot y_2$

If $\lambda = 0.7$, the mixed image is 70% from image 1 and 30% from image 2.

The mixed label says: 70% cat, 30% dog — not a single hard answer.

λ is sampled from $\text{Beta}(\alpha, \alpha)$, where α is a hyperparameter (commonly 0.2 or 0.4).

The deep insight behind Mixup is that it trains the model to behave linearly between data points. A network trained with Mixup learns not just to classify individual examples correctly, but to interpolate smoothly between them. This produces better-calibrated confidence estimates and improved robustness to adversarial perturbations.

Real-World Example: Mixup in Content Moderation

Facebook AI applied Mixup when training models to detect harmful content (violence, nudity, misinformation).

Hard-label training produced overconfident models that were brittle at boundary cases — images that were partially but not fully in violation.

Mixup training improved calibration: the model became appropriately uncertain about edge cases rather than confidently wrong.

This matters enormously in content moderation: false positives remove legitimate content, false negatives allow harm. Calibrated uncertainty enables better human-review routing.

A model that says 'I am 60% confident this violates policy' is more useful than one that blindly says 'violation: 99.9%'.

4.3 CutMix — Teaching Part-Based Recognition

CutMix (Yun et al., 2019) takes a different approach to mixing: instead of blending images pixel-by-pixel, it cuts a rectangular patch from one image and pastes it into another. The label is then mixed proportionally to the area of the patch relative to the total image area.

CutMix Formula

- Given images x_1, x_2 and labels y_1, y_2 :
1. Sample a rectangular patch region R with area A from image x_2 .
 2. Paste the patch: $\tilde{x} = x_1$ with region R replaced by the corresponding region from x_2 .
 3. Mixed label: $\tilde{y} = (1 - A/\text{total}) \cdot y_1 + (A/\text{total}) \cdot y_2$
- If the patch covers 30% of the image, the label is 70% from image 1 and 30% from image 2.
- Unlike Mixup, each pixel belongs entirely to one image — there is no blending artifact.

The deep insight behind CutMix is that it forces the model to recognise objects from partial views — exactly as humans do. A person can identify a dog from seeing only its ear, or a car from seeing only its wheel. Standard training on whole objects can produce models that fail when the object is partially occluded, because they have never been trained on partial views.

Real-World Example: Autonomous Vehicles and Occlusion

Autonomous vehicle perception systems must reliably detect pedestrians even when partially hidden behind parked cars, poles, or other vehicles.

Models trained only on fully visible pedestrians learned to rely on whole-body silhouettes — failing when only legs or a torso were visible.

Training with CutMix forced the model to recognise pedestrian features from partial views: a shoe, an arm, a head above a car hood.

Waymo and other AV companies reported measurable improvements in partially-occluded pedestrian detection after incorporating CutMix-style training.

This is exactly the scenario where CutMix excels: when real deployment conditions involve occlusion that clean training data doesn't capture.

Property	Mixup	CutMix
Image structure	Blend of two images (ghostly, transparent)	Patch from one image pasted into another (no blending artifacts)
Label assignment	Proportional to mixing coefficient λ	Proportional to patch area ratio

Property	Mixup	CutMix
Feature learning	Teaches linear interpolation between classes	Teaches recognition from partial views
Best use case	Classification with similar-looking classes	Detection/classification tasks with occlusion
Training stability	Very stable; easy to tune	Stable; can be more aggressive with patch size

4.4 RandAugment — Simplifying Learned Augmentation

AutoAugment (Cubuk et al., Google Brain, 2018) introduced the idea of learning augmentation policies using reinforcement learning. A controller network tried millions of augmentation combinations and learned which sequences of operations (e.g., rotate → colour jitter → sharpness) improved validation accuracy. The result was powerful — but the search cost millions of GPU hours.

RandAugment (Cubuk et al., 2019) asked a simpler question: do we actually need to learn the policies, or can random selection work just as well? RandAugment randomly selects N augmentation operations from a predefined list and applies them at a uniform magnitude M . With just two hyperparameters (N and M), it achieves performance competitive with AutoAugment at a fraction of the search cost.

The key insight is that augmentation diversity matters more than precisely learning which operations to apply. Randomly drawing from a broad set of transformations exposes the model to a wide variety of distortions — and that diversity is what improves generalisation.

Real-World Example: RandAugment in EfficientNet Training

Google's EfficientNet paper used RandAugment as the augmentation strategy during training. For EfficientNet-B7, they set $N=2$ (two operations per image) and $M=27$ (high magnitude), creating very aggressive augmentation.

This level of augmentation would have destroyed an under-regularised model — but EfficientNet's compound-scaled architecture and dropout handled it well.

RandAugment is now the default augmentation recommendation in PyTorch's torchvision library.

The practical lesson: start with $N=2$, $M=9$, then increase M as your model gets larger and more regularised.

4.5 Test-Time Augmentation (TTA)

Test-Time Augmentation is conceptually simple: instead of running a single forward pass at inference time, run multiple forward passes on different augmented versions of the same image and average the predictions. The most common approach applies horizontal flips, crops, and brightness variations — producing perhaps 5 to 10 predictions per image — then averages the class probability distributions.

The statistical reason TTA works is that each augmented version introduces different noise into the prediction. By averaging many predictions, the random noise cancels out while the consistent signal — the true class — is amplified. This is exactly the same principle as ensemble methods: multiple independent predictions averaged together are more reliable than any single prediction.

Real-World Example: Kaggle Competition Winners

In competitive machine learning (Kaggle competitions), TTA is one of the most reliable free accuracy boosts available.

In the 2022 Kaggle Happywhale competition (identifying individual whale flukes from underwater photographs), the top teams used 10–20x TTA at inference.

TTA consistently adds 0.1–1.5% accuracy at the cost of proportionally more inference time.

This tradeoff is perfect for competitions (where accuracy is everything and latency doesn't matter) and for high-stakes single-prediction tasks like medical screening.

For production APIs with latency requirements, TTA is usually impractical — but for batch processing or offline tasks, it is almost always worth applying.

4.6 Augmentation Technique Comparison

Technique	Mechanism	Accuracy Gain	Training Cost	Best Use Case
Standard (Flip/Crop/Jitter)	Transform individual images	Baseline	Minimal	Always use as the foundation
Mixup	Blend two images and labels proportionally	0.5–1.5%	Low (simple α)	Classification with similar-looking classes; calibration
CutMix	Paste patch from one image into another	0.5–2.0%	Low	Occlusion robustness; object detection
RandAugment	Random N ops at magnitude M	0.5–2.5%	Very Low (just N and M)	General purpose; default for new projects
AutoAugment	Learned augmentation policies	0.5–3.0%	Very High (search cost)	When compute allows search; research settings
TTA	Multiple augmented inference passes averaged	0.1–1.5%	Inference cost only	High-stakes single predictions; competitions

05 — Attention Mechanisms & Transformers

The Transformer architecture (Vaswani et al., 2017) is arguably the most important architectural innovation in the history of deep learning. It introduced a fundamentally new way for neural networks to process sequences — one that has since conquered natural language processing, computer vision, audio, and scientific computing. This section builds a deep conceptual understanding of attention mechanisms from first principles.

5.1 The Problem Attention Was Designed to Solve

Before Transformers, sequence-to-sequence tasks (like machine translation) were dominated by Recurrent Neural Networks and their variants — LSTMs and GRUs. These architectures processed sequences step by step: they read input tokens one at a time, updating an internal hidden state at each step, and produced output tokens one at a time.

This sequential processing created two serious problems. First, there was an information bottleneck: the entire meaning of a long input sequence had to be compressed into a single fixed-size vector (the final hidden state) before the decoder could begin generating output. A 200-word sentence had to fit in a vector of perhaps 512 numbers — an enormous compression that inevitably lost information.

Second, long-range dependencies were difficult to model. Consider the sentence: 'The trophy didn't fit in the bag because it was too big.' What does 'it' refer to — the trophy or the bag? Humans resolve this instantly using the word 'big', which applies to the trophy. But an LSTM has to propagate information about 'trophy' across many time steps to reach 'it' — and gradient signals weaken with each step, making it difficult for the model to learn these long-range relationships.

Real-World Example: Google Translate Before and After Attention

Google Translate used an LSTM encoder-decoder architecture until 2016. The system was competent for short sentences but degraded visibly on longer ones.

The information bottleneck was the culprit: a 100-word sentence had to be compressed into a single encoder vector, inevitably losing information.

In September 2016, Google replaced the LSTM with an attention-based architecture for eight language pairs.

The improvement was dramatic — on long sentences, the average translation quality jumped by what Google described as more than in the previous 10 years of LSTM development.

The key change: instead of looking only at the final encoder state, the decoder could now attend to the most relevant encoder states for each output word.

5.2 Self-Attention — Query, Key, Value

Self-attention is the core operation of the Transformer. It allows every element in a sequence to look at every other element and decide how much attention to pay to it. The mechanism is built around three learned projections of each input: a Query, a Key, and a Value.

The intuition comes from an information retrieval analogy. Imagine a library. Your Query is the question you are looking for an answer to. The Keys are the index labels on every book in the library. The Values are the actual content of those books. Self-attention computes the similarity between your Query and every Key in the library, uses those similarities as weights, and returns a weighted combination of all the Values. The most relevant books contribute the most to your answer.

📖 Scaled Dot-Product Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

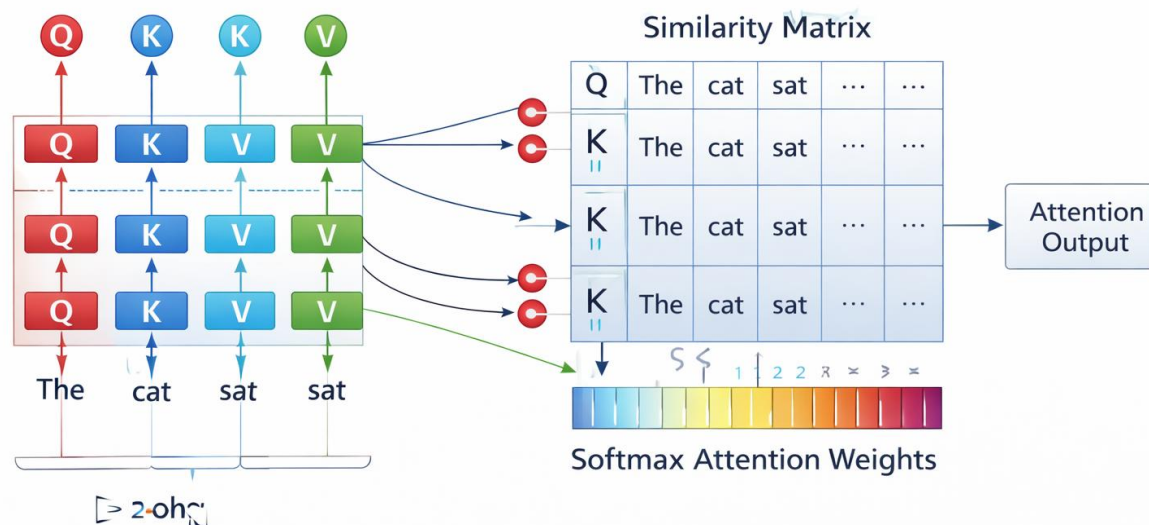
Step 1: Compute similarity scores: $S = Q \cdot K^T$ (dot product of Query with all Keys)

Step 2: Scale scores: $S = S / \sqrt{d_k}$ (prevents softmax saturation for large embedding dimensions)

Step 3: Normalise to weights: $W = \text{softmax}(S)$ (weights sum to 1; most similar Keys get highest weight)

Step 4: Weighted sum of Values: $Z = W \cdot V$ (output is a mixture of all Values, weighted by relevance)

Why $\sqrt{d_k}$? For large d_k , dot products grow large $\rightarrow$ softmax saturates $\rightarrow$ near-zero gradients $\rightarrow$ slow learning. Scaling fixes this.



🔍 Worked Example: 'The cat sat on the mat'

Consider the word 'sat'. Its Query asks: 'What context is most relevant to understanding me?'

The Keys of all other words respond: 'cat' says 'I am the subject of the verb', 'mat' says 'I am the location'.

After softmax, 'cat' and 'mat' might get weights like 0.4 and 0.3, while 'the', 'on' get weights like 0.1.

The Value output for 'sat' is a blend: 40% cat-information, 30% mat-information, 10% 'the'-information, etc.

This output representation of 'sat' now contains contextual information about what sat (the cat) and where (the mat).

Critically, this happens in parallel for all words simultaneously — not sequentially like an RNN.

🌐 Real-World Example: How GPT-4 References Earlier Code

When GPT-4 generates code inside a long function, it needs to reference variable names defined hundreds of tokens earlier.

An LSTM would struggle: the gradient signal from a variable definition at token 50 is nearly gone by token 600.

Self-attention has no such decay. Token 600's Query can directly attend to token 50's Key with full strength — distance is irrelevant.

This is why GPT-4 can maintain consistent variable names, function signatures, and logical structure across very long code blocks.

Every output token can, in principle, directly attend to every input token — regardless of how far apart they are.

5.3 Multi-Head Attention — Multiple Perspectives Simultaneously

Single-head attention computes one attention pattern — one way of relating tokens to each other. But language and images contain many different kinds of relationships simultaneously. In a sentence, some words are related grammatically (subject-verb agreement), others semantically (synonym or antonym), others by coreference (pronoun resolution), others by proximity.

Multi-head attention runs several attention operations in parallel, each with its own independent Query, Key, and Value projections. Each 'head' learns to attend to a different type of relationship. The outputs of all heads are concatenated and linearly projected to the final output dimension.

Multi-Head Attention Formula

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) \cdot W^O$$

$$\text{Where: } \text{head}_i = \text{Attention}(Q \cdot W_iQ, K \cdot W_iK, V \cdot W_iV)$$

Each head has its own learned projection matrices: W_iQ , W_iK , W_iV

W^O is the output projection matrix that combines all heads

Typical configurations: $h=8$ heads in BERT-base, $h=16$ in BERT-large, $h=32+$ in GPT-3/4

Real-World Example: What Different Attention Heads Learn

Researchers at OpenAI and academic institutions have visualised what individual attention heads in BERT learn.

Head 1 in many models consistently attends to the next token — learning local positional structure.

Head 4 in some layers learns to track subject-verb agreement — a grammatical, long-range syntactic relationship.

Head 7 in some configurations learns coreference resolution — consistently linking pronouns ('it', 'they') to their antecedents.

Head 11 in certain layers learns to track named entities — attending from any mention of 'Microsoft' to previous mentions.

These specialisations emerge purely from training on text — the model discovers that different relationship types are useful, without being told to.

'reads' the encoded input: each decoder position attends to the most relevant positions in the encoder's representation.

Positional Encoding — Teaching Order Without Recurrence

Unlike RNNs, which process sequences step-by-step and inherently encode position through their sequential structure, Transformers process all tokens in parallel. Without additional information, a Transformer would treat 'the cat sat' and 'sat cat the' identically — the order of tokens would be invisible.

Positional encodings solve this by adding a position-dependent signal to each token's embedding before it enters the Transformer. The original paper used sinusoidal functions — sine and cosine waves at different frequencies — to encode position. Modern models typically use learned positional embeddings instead, allowing the model to discover the most useful positional representation for its task.

 Real-World Example: Google Translate — Encoder Reads, Decoder Writes

When translating 'Le chat s'est assis sur le tapis' to English, the encoder reads the entire French sentence in parallel.

Each French word's final representation contains information from all other words — 's'est assis' knows that 'le chat' is the subject.

The decoder generates English one word at a time. When generating 'sat', the decoder's cross-attention query looks at the encoder's 's'est assis' representation most strongly.

When generating 'mat', the decoder's cross-attention correctly attends to 'tapis' — even though 'mat' and 'tapis' are in different positions in their respective sentences.

This flexible word reordering is what makes Transformer-based translation so much better at handling languages with different syntactic structures.

5.5 BERT vs GPT — A Deep Conceptual Comparison

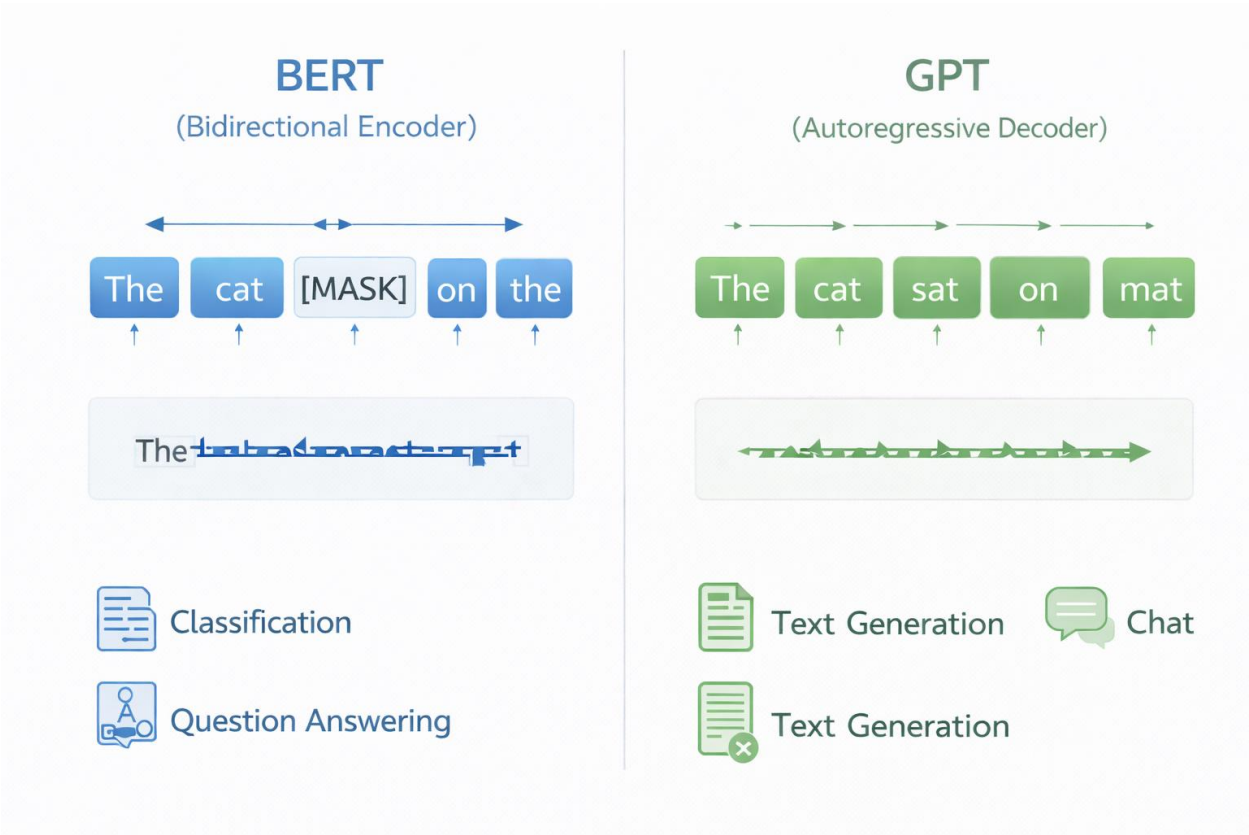
BERT (Bidirectional Encoder Representations from Transformers, Devlin et al., 2018) and GPT (Generative Pretrained Transformer, Radford et al., 2018) share the Transformer building block but make opposite architectural choices — choices that reflect fundamentally different views of what language understanding means.

BERT uses the Transformer encoder. During pretraining, it randomly masks 15% of input tokens and trains the model to predict the masked words using context from both directions — words before and after the mask. This bidirectional context makes BERT excellent at understanding: it can see the full context around any position and produce rich representations for downstream understanding tasks like classification and question answering.

GPT uses the Transformer decoder with causal masking. It predicts the next word given only the words that came before — it never looks ahead. This constraint is not a limitation but a design choice: it means the model can be used for open-ended text generation, because generation inherently requires producing tokens one at a time without knowing what comes next.

Property	BERT	GPT
Architecture	Transformer Encoder only	Transformer Decoder only (causal)
Attention direction	Bidirectional — sees all tokens	Causal — only sees past tokens
Pretraining objective	Masked Language Model (fill in blanks)	Causal Language Model (predict next word)

Property	BERT	GPT
Strength	Understanding and classification tasks	Text generation and completion tasks
Best tasks	Search, NER, question answering, sentiment	ChatGPT, code generation, creative writing
Famous models	BERT, RoBERTa, DistilBERT, ALBERT	GPT-2, GPT-3, GPT-4, LLaMA, Mistral



Real-World Example: BERT Powers Google Search

In October 2019, Google announced that BERT was deployed in Google Search to better understand search query intent.

The example they gave: 'Can you get medicine for someone pharmacy'. Before BERT, Google interpreted 'for someone' as irrelevant and returned generic pharmacy results.

BERT's bidirectional understanding captured the full intent: someone wants to pick up a prescription on behalf of another person.

After BERT, Search returned results specifically about prescription pickup policies — the right answer.

GPT-4, by contrast, powers ChatGPT: it generates responses word by word, creating coherent multi-paragraph text, code, and structured outputs.

Modern production AI systems often use both: BERT-style encoding for retrieval and understanding, GPT-style decoding for generation (the RAG architecture).

5.6 Vision Transformer (ViT) — Attention for Images

Having mastered self-attention in language, researchers asked: can the same mechanism work for images? Vision Transformer (ViT) answered with a resounding yes — by treating image patches as the tokens.

The [CLS] token is a special learnable embedding prepended to the sequence of patch embeddings. It has no corresponding image region — its sole role is to accumulate information from all patches through the self-attention layers. By the final layer, the [CLS] token's representation has attended to every patch in the image and summarised the global visual content. A linear classifier applied to the [CLS] token output produces the final prediction.

The global nature of self-attention is ViT's defining advantage. A CNN's receptive field grows gradually through the network — early layers see only small patches, later layers see larger regions, and only the deepest layers see the full image. A ViT's first attention layer already relates every patch to every other patch — giving it immediate access to global context.



Real-World Example: Google Lens and Scene Understanding

Google Lens uses ViT-based models to understand entire scenes, not just identify individual objects.

When you point Lens at a restaurant menu, it doesn't just recognise individual words — it understands the layout, the relationships between dishes and prices, and the overall context.

This scene-level understanding benefits from ViT's global attention: the [CLS] token attends to text patches, image patches of dishes, layout structure, and surrounding context simultaneously.

A CNN would process these elements locally and gradually, potentially missing relationships between distant parts of the image.

ViT's ability to immediately relate any patch to any other patch makes it naturally suited to tasks that require holistic scene understanding.

06 — Advanced Training Techniques

Beyond architecture and data, the techniques used during training have a profound impact on model quality, training stability, and computational cost. This section covers five advanced training techniques that are now standard in production deep learning pipelines.

6.1 Gradient Clipping — Taming Explosive Updates

During backpropagation, gradients flow backward through the network, updating each layer's weights proportionally to their gradient magnitude. In well-behaved training, gradients are moderate and updates are stable. But in some architectures — particularly deep networks, recurrent networks, and Transformers — gradient magnitudes can occasionally spike to enormous values, causing weights to change drastically in a single step. This is called the exploding gradient problem.

Gradient clipping addresses this by setting a maximum allowable gradient norm. If the global gradient norm (the overall magnitude of all gradients combined) exceeds a threshold, all gradients are scaled down proportionally so that the norm equals the threshold. This is not the same as truncating — it preserves the direction of the gradient update while limiting its magnitude.

Gradient Clipping Formula

If $\|g\| > \text{max_norm}$, scale all gradients:

$$g_{\text{clipped}} = g \times (\text{max_norm} / \|g\|)$$

Where $\|g\|$ is the L2 norm of all gradients concatenated into a single vector.

The direction of the gradient is preserved; only the magnitude is reduced.

Typical max_norm values: 1.0 for Transformers, 5.0 for RNNs.

Real-World Example: Stabilising GPT Training

Early GPT training runs at OpenAI were notoriously unstable — the loss would suddenly spike to infinity after thousands of training steps, requiring expensive restarts.

Gradient explosions were a primary culprit: a single bad batch caused gradients to spike, destroying the weights built up over days of training.

Gradient clipping ($\text{max_norm} = 1.0$) became a non-negotiable standard in all large language model training pipelines.

A helpful mental model: if gradient descent is walking downhill, gradient clipping ensures no single step can be longer than a metre — you can never accidentally step off a cliff.

Without clipping, a single catastrophic batch can undo millions of gradient steps of accumulated learning.

6.2 Label Smoothing — Calibrating Confidence

Standard cross-entropy loss trains models against hard labels: the correct class has probability 1.0 and all other classes have probability 0.0. This encourages models to assign near-zero probability to all wrong classes — but this extreme confidence is often unwarranted and hurts generalisation.

Label smoothing softens the target distribution: instead of a probability of 1.0 for the correct class, the model is trained toward a distribution where the correct class has probability $(1-\epsilon)$ and the remaining

probability ϵ is spread uniformly across all K classes. With $\epsilon = 0.1$ and $K = 1000$ classes, the correct class target becomes 0.9 and each wrong class gets 0.0001.

Label Smoothing Formula

Hard label: $y_{\text{hard}} = [0, 0, 1, 0, 0, \dots]$ (one-hot: 1 for correct class, 0 for all others)

Smooth label: $y_{\text{smooth}} = (1-\epsilon) \cdot y_{\text{hard}} + \epsilon/K$

For $\epsilon = 0.1$ and $K = 1000$:

Correct class target: $(1-0.1) + 0.1/1000 = 0.9001$

Each wrong class target: $0.1/1000 = 0.0001$

The model is penalised for being too confident, even when it is correct.

Real-World Example: Label Smoothing in Medical AI Safety

A deep learning model diagnosing skin lesions was trained with hard labels. It achieved 94% accuracy but was dangerously overconfident.

On ambiguous cases — lesions that even expert dermatologists disagreed on — the model said 'benign: 99.9%' rather than expressing appropriate uncertainty.

When clinicians saw high-confidence predictions, they deferred to the AI even when their own clinical judgment suggested uncertainty.

After retraining with label smoothing ($\epsilon = 0.1$), the model's confidence on ambiguous cases dropped to 65–75% — triggering human review for those cases.

Label smoothing turned an overconfident model into a better collaborator: it knew when to flag uncertainty rather than asserting false certainty.

6.3 Mixed Precision Training — Speed Through Numerical Precision

Modern deep learning models use 32-bit floating point (FP32) by default for storing weights and gradients. FP32 provides excellent numerical precision but is expensive: it requires 4 bytes per number, and operations on FP32 are slower than on lower-precision formats on modern GPUs.

Mixed precision training uses 16-bit floating point (FP16 or the newer BF16) for the forward and backward passes — computing activations and gradients in half precision — while maintaining a master copy of weights in FP32 for the weight update step. On Nvidia A100 GPUs and similar hardware, FP16 matrix multiplications run 2–4× faster and use half the memory compared to FP32.

The key challenge is numerical underflow: very small gradient values can round to zero in FP16, causing the update to be silently lost. Loss scaling solves this: before the backward pass, the loss is multiplied by a large scaling factor (e.g., 65536). The gradients are then proportionally larger and stay above the FP16 underflow threshold. After computing gradients, they are divided back by the scaling factor before the weight update.

Real-World Example: Training GPT-3 with Mixed Precision

GPT-3 has 175 billion parameters. In FP32, storing the parameters alone requires 700GB — far exceeding the memory of any single GPU.

Mixed precision training was not optional — it was the only viable approach. FP16 halved the memory requirement and doubled throughput on the A100 GPUs used for training.

Without mixed precision, training GPT-3 would have taken roughly twice as long and required twice the hardware — an additional tens of millions of dollars.

Mixed precision is now enabled by default in every major deep learning framework: PyTorch's `torch.amp.autocast` and TensorFlow's `tf.keras.mixed_precision`.

For individual researchers with a single GPU, mixed precision typically allows doubling batch size or using a 2× larger model within the same memory budget.

6.4 Gradient Accumulation — Simulating Large Batches

Training stability and generalisation quality both benefit from large batch sizes. The gradient computed from a large batch is a better estimate of the true gradient of the loss over the full dataset — it averages out more noise. Large batches also enable more stable learning rate schedules.

However, GPU memory directly limits batch size. Doubling the batch size roughly doubles the memory required for activations during the forward pass. Gradient accumulation provides a way to simulate large batch training without the memory cost: run multiple small forward and backward passes, accumulate the gradients across those passes without updating weights, and only perform the weight update after accumulating the equivalent of one large batch.

Gradient Accumulation Formula

Effective batch size = `micro_batch_size` × `accumulation_steps`

Example: `micro_batch_size` = 8, `accumulation_steps` = 32
→ Effective batch size = 256

Process:

1. For each accumulation step: forward pass → compute loss / `accumulation_steps` → backward pass.
2. After all accumulation steps: call `optimizer.step()` and `optimizer.zero_grad()`.
3. The accumulated gradients represent the gradient of the large effective batch.

Real-World Example: Research Parity on Consumer Hardware

The BERT paper trained with a batch size of 256 sequences using 64 TPU chips — hardware available to essentially no academic researchers.

A researcher with a single 8GB GPU can only fit batch size 8 in memory.

With gradient accumulation of 32 steps, that researcher achieves an effective batch size of 256 — matching the original BERT training conditions.

The training takes 32× more wall-clock time per effective update, but produces statistically equivalent results to large-batch training.

Gradient accumulation democratised large-model training: academic groups can reproduce state-of-the-art results with commodity hardware.

6.5 Exponential Moving Average (EMA) of Weights

The weights at the end of training are the result of the final gradient update — a single noisy step. Model weights oscillate throughout training as each batch pushes them in slightly different directions. The weights at the very last step may happen to be slightly off the optimal position due to the noise in the last batch.

EMA maintains a smoothed version of the weights that slowly tracks the training weights over time. After each update step, the EMA weights are updated as a weighted average of the current EMA weights and the new training weights. With a high momentum coefficient α (e.g., 0.999), the EMA weights change very slowly — they represent a smoothed average over the last $\sim 1/(1-\alpha)$ training steps.

EMA Formula

After each training step:

$$\theta_EMA = \alpha \cdot \theta_EMA + (1-\alpha) \cdot \theta_train$$

With $\alpha = 0.999$:

θ_EMA effectively averages over ~ 1000 recent training steps.

θ_EMA changes at 0.1% of the rate of the training weights.

At inference time, use θ_EMA — not the raw training weights.

EMA weights consistently generalise better than the final training step weights.

Real-World Example: EMA in Stable Diffusion

Stable Diffusion (Rombach et al., 2022) uses EMA weights for the final deployed model — the weights actually used to generate images are not the raw training weights.

The training weights are noisy and optimised for a specific batch composition at the last training step.

The EMA weights represent a smooth average of the model's state over thousands of recent training steps — much more stable.

Users of Stable Diffusion consistently rated EMA-weight images as higher quality and more coherent than the raw training-weight images.

The analogy: in finance, a 30-day moving average of a stock price is much smoother and more predictive than today's closing price alone.

07 — Model Compression & Deployment

A model that achieves 95% accuracy on a research benchmark but requires 5 seconds per inference on a smartphone is useless in production. Model compression is the discipline of reducing model size and increasing inference speed — ideally with minimal accuracy loss. This section covers the four main compression techniques and the universal deployment format ONNX.

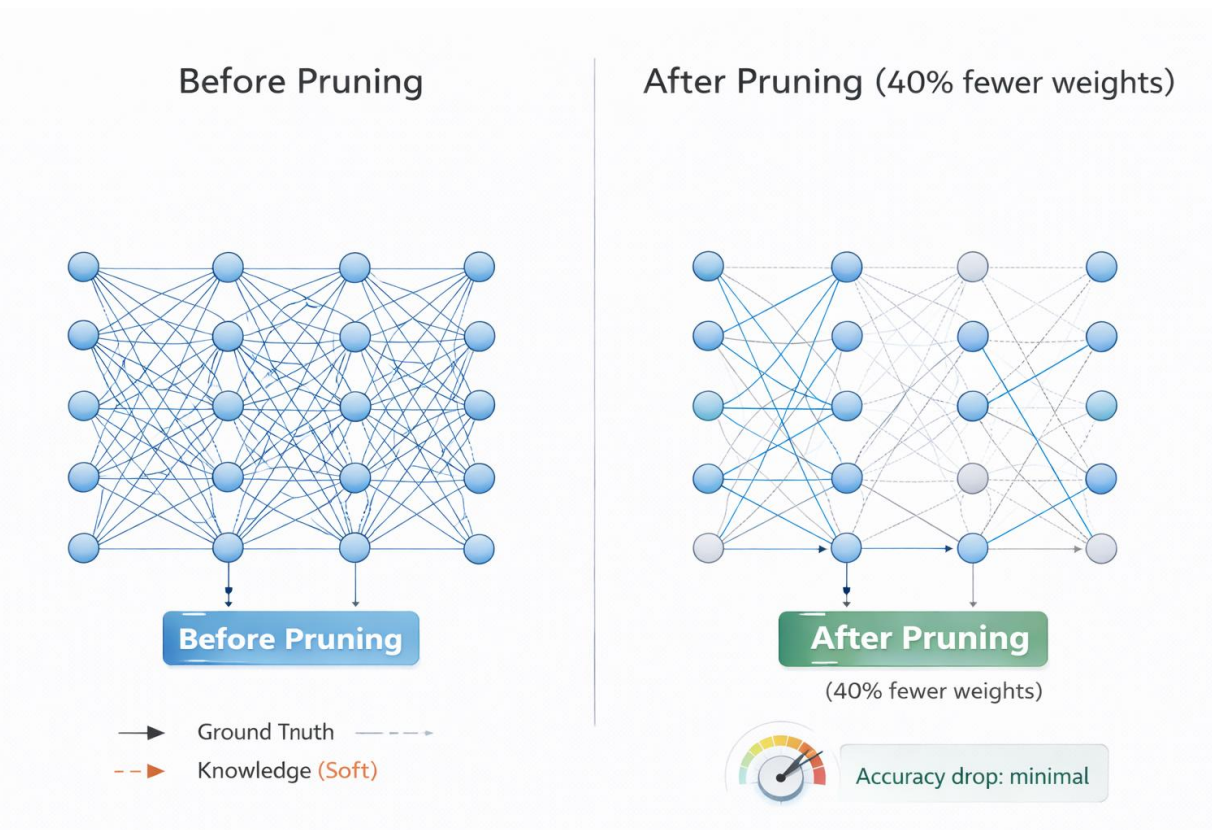
7.1 The Production Trilemma

Every deployment decision involves tradeoffs between three competing objectives: accuracy (how often the model is correct), speed (latency from input to output), and size (memory and storage footprint). Improving any one dimension typically comes at the cost of the others.

The stakes are high in practice. A 50ms latency target for a mobile app is non-negotiable — users abandon apps that feel slow. A 50MB model size limit for an iOS app is enforced by the App Store. These hard constraints force engineers to find the highest accuracy within the given speed and size budget.

7.2 Pruning — Removing What Doesn't Matter

Pruning is based on a powerful insight: large neural networks are massively overparameterised. They contain many more parameters than they theoretically need to represent the learned function. Many of these parameters have very small magnitudes — they contribute almost nothing to the output. Removing them (setting them to zero) simplifies the model without significantly hurting accuracy.



Magnitude pruning removes weights whose absolute value falls below a threshold. The intuition is that a weight with magnitude 0.001 changes its neuron's output by almost nothing — removing it is a tiny perturbation to the model's function. A weight with magnitude 5.0 has substantial influence and should be kept.

Structured pruning removes entire neurons, filters, or attention heads — rather than individual weights. This produces a genuinely smaller network (fewer matrix multiplications) and achieves real speedups on standard hardware. Unstructured pruning (removing individual weights) creates sparse weight matrices, which are smaller in storage but do not necessarily run faster on GPUs without specialised sparse computation support.

Real-World Example: Meta's Content Moderation Models

Meta runs content moderation models on billions of posts per day across Facebook, Instagram, and WhatsApp.

At that scale, even a 10% reduction in inference cost saves millions of dollars annually in compute.

Meta's engineers pruned their content moderation models by 40% (removing 40% of parameters) with less than 1% accuracy drop.

The pruned models ran measurably faster due to structured pruning of entire attention heads — standard matrix multiplication code became cheaper without any hardware modification.

This story illustrates why compression is not academic: at Facebook's scale, pruning is a direct business requirement.

7.3 Quantization — Reducing Numerical Precision

Quantization reduces the numerical precision used to represent weights and activations. A standard model uses 32-bit floating point (FP32) — each number has 4 bytes of precision. Quantizing to 8-bit integers (INT8) reduces each number to 1 byte: 4× smaller, with 2–4× faster inference on hardware with INT8 support.

The quantization process maps the continuous range of FP32 values to the discrete range of INT8 integers using a scale factor and zero point. The scale maps the dynamic range of the weights to the INT8 range [-128, 127]; the zero point handles asymmetric distributions.

Post-Training Quantization (PTQ) applies quantization to an already-trained model without retraining. It is fast and easy but can lose accuracy on models where the weight distributions are poorly suited to INT8 representation. Quantization-Aware Training (QAT) simulates quantization during training — the model learns to minimise quantization errors as part of its optimisation. QAT produces better accuracy than PTQ but requires retraining.

Real-World Example: Apple Face ID and INT8 Quantization

Apple's Face ID must authenticate faces in under 300 milliseconds on the iPhone's Neural Engine — with no internet connection.

The underlying neural network runs as a quantized INT8 model compiled for the Neural Engine's optimised matrix multiplication units.

INT8 quantization reduces the model to one-quarter of the FP32 size and runs 2–4× faster — meeting the strict latency budget.

Apple's Quantization-Aware Training ensures that Face ID's accuracy is not sacrificed: the model was trained knowing it would be quantized.

Without quantization, Face ID's network would be too large for on-device memory and too slow for the sub-second authentication experience.

7.4 Knowledge Distillation — Learning from a Teacher's Uncertainty

Knowledge distillation (Hinton et al., 2015) is a technique for training a small, efficient student model to mimic a large, accurate teacher model. The student does not just learn from the hard ground-truth labels — it learns from the teacher's output probability distributions, which contain far more information.

Consider a teacher model classifying an image of a cat. Its output might be: cat 87%, tiger 8%, lion 4%, dog 1%. These soft probabilities reveal the teacher's understanding of visual similarity — the image looks slightly like a tiger and a lion, not at all like an airplane. This information is invisible in the hard label 'cat: 1.0'.

The temperature parameter T controls how soft these distributions are. At $T=1$, the teacher's probabilities are used as-is. At $T=4$, the distribution is softened (the differences between class probabilities are reduced), making the subtle relationships between similar classes more visible to the student. After training, the student is used at $T=1$ for normal inference.

Knowledge Distillation Loss

$$L_{\text{total}} = \alpha \cdot L_{\text{CE}}(\text{hard labels}) + (1-\alpha) \cdot L_{\text{KL}}(\text{teacher soft labels}, T)$$

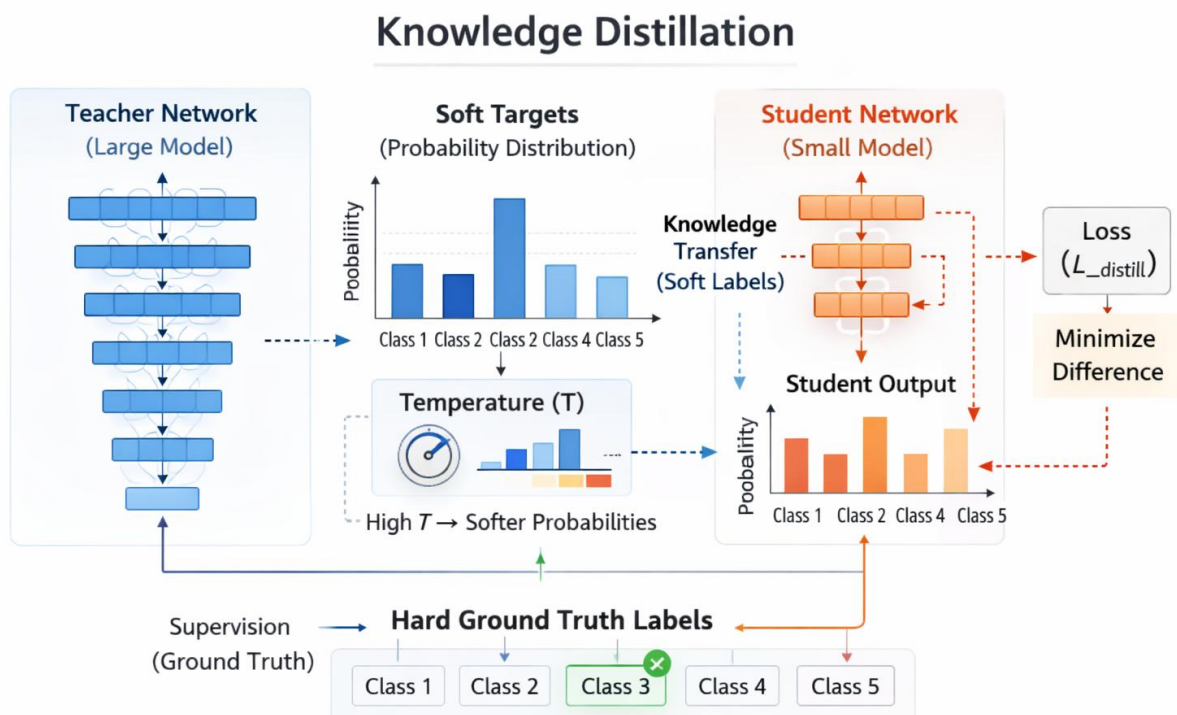
L_{CE} : Standard cross-entropy with ground-truth hard labels (1-hot)

L_{KL} : KL-divergence between student output (at temperature T) and teacher output (at temperature T)

α : Weight controlling the balance. Commonly 0.1–0.3 (mostly learn from teacher's soft labels)

T : Temperature. Higher T = softer teacher distributions = more information about class relationships

Key insight: At $T > 1$, even the teacher's near-zero probabilities carry signal about which classes are visually similar.



Real-World Example: DistilBERT — 40% Smaller, 60% Faster

BERT-base has 110 million parameters and requires substantial compute for inference at scale.

Hugging Face trained DistilBERT by distilling BERT into a 6-layer student (down from 12 layers) — 40% fewer parameters.

The student was trained using BERT's soft output probabilities, not just the hard training labels. It learned BERT's nuanced uncertainty about token classifications.

Result: DistilBERT retains 97% of BERT's performance on GLUE benchmarks while running 60% faster and using 40% less memory.

DistilBERT powers Hugging Face's free inference API — the scale savings from distillation make the free tier economically viable.

Google's Gemini Nano (the on-device version running on Pixel phones) is similarly a distilled version of Gemini Pro.

7.5 ONNX — The Universal Deployment Format

Open Neural Network Exchange (ONNX) is an open format for representing machine learning models in a framework-agnostic way. A model trained in PyTorch can be exported to ONNX and then loaded and run by any ONNX-compatible runtime — including ONNX Runtime (for CPU and GPU), TensorRT (NVIDIA GPUs), CoreML (Apple), and TFLite (mobile).

ONNX solves a real problem: research and production environments often use different frameworks. A research team trains in PyTorch; the production engineering team deploys with TensorRT for maximum GPU efficiency. Without ONNX, converting between frameworks requires manual reimplementations and verification. ONNX provides a standardised intermediate representation — a computational graph in a common format.

The ONNX Runtime also applies graph optimisations automatically: constant folding, operator fusion (combining sequential operations into a single optimised kernel), and memory planning. These optimisations are independent of the framework used for training and often provide additional speedups beyond what PyTorch's eager mode achieves.

Real-World Example: Airbnb's Framework-Agnostic Production Pipeline

Airbnb's data scientists train recommendation and search ranking models in PyTorch — the most productive research environment.

The production engineering team needs to serve these models in a scalable, optimised inference server independent of the training framework.

Airbnb exports trained PyTorch models to ONNX, then deploys them via ONNX Runtime in their production serving infrastructure.

The result: data scientists can use PyTorch freely without worrying about production compatibility; engineers can optimise the deployment without modifying training code.

ONNX is the contractual interface between research and engineering — each team works in their preferred environment, meeting in the middle.

Deployment Target	Runtime	Best For	Key Advantage
CPU servers	ONNX Runtime	General deployment, low cost	Universal; no specialised hardware needed
NVIDIA GPUs	TensorRT	Real-time inference at scale	GPU kernel fusion; maximum throughput
Apple devices (iPhone, Mac)	CoreML	iOS and macOS apps	Neural Engine acceleration; private on-device
Android/embedded	TFLite	Mobile and IoT devices	Tiny binary size; ARM optimised
Browser (WebAssembly)	ONNX.js / TFJS	Client-side ML in web apps	No server needed; runs in the browser

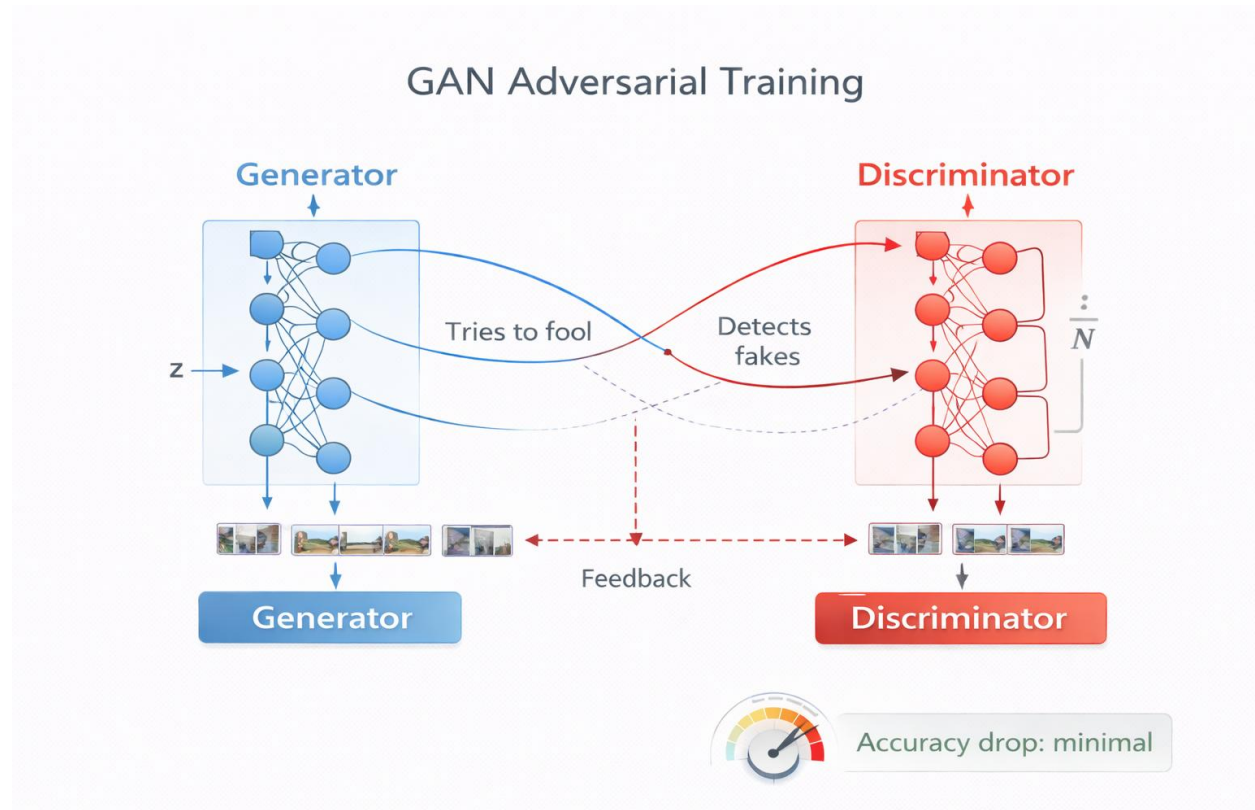
08 — Generative Models Overview

Generative models learn to produce new data — images, text, audio, molecules — that resemble the training data distribution. They are one of the most exciting and rapidly advancing areas in deep learning. This section covers the three dominant generative paradigms: GANs, VAEs, and diffusion models.

8.1 GANs — The Adversarial Game

Generative Adversarial Networks (GANs, Goodfellow et al., 2014) introduced a fundamentally new training paradigm: pit two networks against each other. The Generator takes random noise as input and produces synthetic data (e.g., fake images). The Discriminator takes either real data from the training set or fake data from the Generator and tries to classify each as real or fake.

The adversarial game is defined by their competing objectives: the Generator wants to produce fakes that fool the Discriminator; the Discriminator wants to correctly identify all fakes. As training progresses, the Generator gets better at producing realistic data, and the Discriminator gets better at detecting subtler flaws. In theory, training converges when the Generator produces data indistinguishable from real data.



GAN Minimax Objective

$$\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(z)))]$$

$D(x)$: Discriminator's probability that real sample x is real. D wants this high.

$G(z)$: Generator's output from noise z . D wants $D(G(z))$ low; G wants it high.

Training alternates: update D for k steps, then update G for 1 step.

Equilibrium: G produces samples from the true data distribution; D outputs 0.5 (cannot distinguish real from fake).

In practice, GAN training is notoriously difficult. Two failure modes are common. Mode collapse occurs when the Generator discovers a small set of outputs that consistently fool the Discriminator — and stops producing diverse samples. Instead of generating the full variety of human faces, it might produce only one type of face endlessly. Vanishing gradients occur when the Discriminator becomes too good too quickly: if it can perfectly distinguish real from fake, its gradient signal to the Generator becomes zero — the Generator receives no useful learning signal.

GAN Variant	Key Innovation	Best Use Case
DCGAN (2015)	Replaced fully connected layers with convolutions	Image generation; the first stable GAN architecture
StyleGAN / StyleGAN2	Learned per-layer style conditioning; noise injection	Photorealistic face generation; style transfer
CycleGAN (2017)	Unpaired image-to-image translation with cycle consistency	Domain transfer: photos ↔ paintings, summer ↔ winter
Pix2Pix (2017)	Paired image-to-image with conditional GAN	Image editing: sketches → photos, maps → aerial
BigGAN (2018)	Large-scale class-conditional generation	High-resolution class-conditional image synthesis

 Real-World Example: StyleGAN2 and Synthetic Training Data

NVIDIA's StyleGAN2 produces photorealistic human faces that do not belong to real people — verified by the website 'thispersondoesnotexist.com'.

These synthetic faces are used to create training data for face recognition systems, bypassing privacy concerns around collecting real face data.

Deepfake detection researchers use GANs to generate synthetic deepfakes at scale — training detection systems on synthetic positives is far more scalable than collecting real deepfakes.

The fashion industry uses GANs to generate synthetic models wearing clothing — reducing the cost of product photography.

However, GANs are increasingly being replaced by diffusion models for high-quality generation tasks due to their greater training stability.

8.2 Variational Autoencoders — Structured Latent Space

Variational Autoencoders (VAEs, Kingma & Welling, 2013) take a probabilistic approach to generation. Rather than learning to map inputs to a single point in a latent space (as a standard autoencoder does), a VAE learns to map each input to a distribution — characterised by a mean μ and a standard deviation σ . Samples are drawn from this distribution to produce the latent code used for decoding.

This seemingly small change has a profound consequence: the learned latent space is smooth and continuous. Because each training example is mapped to a distribution rather than a point, nearby regions

of the latent space correspond to semantically similar data. You can interpolate between two points in the latent space and decode each intermediate point — the result is a smooth morphing between the two original examples.

VAE ELBO Loss

$$L_VAE = L_reconstruction + L_KL$$

$L_reconstruction = E[\log p(x | z)]$ — how well the decoder reconstructs the input from z

$L_KL = KL(q(z|x) || p(z))$ — how close the latent distribution is to a standard Normal

The KL term is the key: it regularises the encoder to produce well-organised, continuous latent space. Without KL, the encoder collapses to point estimates and the latent space becomes irregular.

Reparameterisation trick: $z = \mu + \sigma \cdot \epsilon$, where $\epsilon \sim N(0,1)$

This allows gradients to flow through the sampling step — otherwise backpropagation would be impossible.

Real-World Example: Drug Discovery with VAEs

Pharmaceutical companies face a nearly impossible search problem: find drug molecules that bind to a specific protein target from a search space of 10^{60} possible molecules.

Traditional approaches: synthesise candidate molecules one at a time and test them — extremely slow and expensive.

VAE-based drug discovery: train a VAE on known drug molecules. The VAE learns a smooth latent space where nearby points correspond to chemically similar molecules.

Researchers can then optimise in the continuous latent space (using gradient-based methods) toward molecules with desired properties — then decode back to actual molecular structures.

AstraZeneca and other pharmaceutical companies have used VAE-based generative models to identify novel drug candidates for targets including COVID-19 proteins.

8.3 Diffusion Models — The Denoising Revolution

Diffusion models (Ho et al., 2020; Song et al., 2020) have overtaken GANs as the dominant paradigm for high-quality image generation. They are more stable to train, produce more diverse outputs, and can be more easily conditioned on text or other inputs.

The forward process gradually destroys an image by adding Gaussian noise at each of T steps. After T steps (typically 1000), the image has been transformed into pure random noise — visually indistinguishable from random static. The model is then trained to reverse this process: given a noisy image at step t , predict the slightly less noisy version at step $t-1$.

The key insight is that this denoising task is well-defined and stable. At each step, the model only needs to predict a small amount of noise to remove — the step-by-step approach breaks the hard problem of generating complex images into a sequence of much easier denoising steps. The loss function is a simple mean-squared error between the predicted noise and the actual noise added — no adversarial training, no mode collapse.

Why Diffusion Beats GANs for Quality

GAN training: adversarial loss — fragile, prone to collapse, hard to tune.

Diffusion training: simple MSE loss — stable, reliable, easy to scale.

GANs suffer from mode collapse: the generator exploits the discriminator, reducing diversity.

Diffusion models have no such failure mode: each training step is independent and well-defined.

Diffusion models can be easily conditioned: text embeddings, class labels, depth maps can all guide the denoising process.

The price: diffusion requires hundreds of denoising steps at inference — much slower than a single GAN forward pass.

Real-World Example: Stable Diffusion and the Image Generation Revolution

Stable Diffusion (Rombach et al., 2022) combined diffusion models with a key efficiency innovation: run the diffusion process in a compressed latent space, not in pixel space.

The VAE encoder compresses a 512×512 image to a 64×64 latent representation. The diffusion U-Net denoises in this small latent space. The VAE decoder expands back to pixel space.

This reduced the compute cost by $64 \times$ compared to pixel-space diffusion — making high-quality image generation practical for consumer hardware.

Text conditioning is achieved through CLIP text embeddings injected into the U-Net via cross-attention layers — the model learns to associate visual concepts with text descriptions.

DALL·E 3, Midjourney, Adobe Firefly, and virtually all major commercial image generation tools are built on diffusion architectures.

Beyond images: diffusion has been adapted for audio (AudioLDM), video (Sora), 3D models, and protein structure prediction (AlphaFold-3).

8.4 GAN vs VAE vs Diffusion — Comparison

Property	GAN	VAE	Diffusion
Training stability	Low — adversarial; prone to collapse	High — stable ELBO objective	Very High — simple MSE loss
Sample quality	High — sharp, realistic	Medium — often blurry	Very High — SOTA quality
Sample diversity	Low — mode collapse risk	High — structured latent space	Very High — no collapse
Training speed	Moderate	Fast	Slow — 1000-step training
Inference speed	Very Fast — single forward pass	Fast — single encode-decode	Slow — 50–1000 denoising steps
Controllability	Moderate — with conditioning	Good — latent interpolation	Excellent — text, class, image guided

Property	GAN	VAE	Diffusion
Best use case	Style transfer, image-to-image	Drug discovery, anomaly detection	Text-to-image, video, audio generation

09 — Hands-On Lab: Face Recognition System

In this lab you will build a complete face recognition system from scratch using transfer learning — exactly the principles taught in this session. The project uses a dataset of 5 people with 20 images each (100 images total) and demonstrates why domain-specific pretrained models dramatically outperform general-purpose ones on small datasets.

Project Overview — Why This Project?

With only 20 images per person, training a deep network from scratch is impossible — it will overfit immediately. This is precisely the scenario where transfer learning is not just helpful but essential. This project demonstrates three core Day 9 lessons simultaneously:

- **Transfer Learning in action:** A model pretrained on millions of face images is reused; you only train a small classifier on top.
- **Domain-specific pretraining:** A model trained on faces (FaceNet) dramatically outperforms a model trained on general images (ResNet-50 on ImageNet).
- **Feature extraction strategy:** With only 100 images total, feature extraction (frozen backbone) is the only viable approach — fine-tuning would overfit.

Why FaceNet? — Model Selection Rationale

Before building anything, we must justify our model choice. With only 20 images per person, the selection is critical. Here is why each option performs the way it does:

Approach	Accuracy (100 imgs)	Why It Performs This Way	Verdict
CNN from Scratch	40–55%	No prior knowledge of faces. 100 images far too few to learn edges, textures, and face geometry from nothing. Massive overfitting.	✗ Don't Use
ResNet-50 (ImageNet)	70–82%	Good general visual features (edges, textures) but never trained on faces. Must adapt general features to face identity — harder with 20 images.	⚠ Acceptable
FaceNet (face-pretrained)	92–97%	Pretrained on millions of face images. Already understands face geometry, lighting variation, and identity. Only the final classifier needs training.	☑ Best Choice
ArcFace (face-pretrained)	94–98%	Uses angular margin loss for tighter face embedding clusters. Marginally better than FaceNet but more complex to set up.	☑ Best Choice

💡 Why Domain-Specific Pretraining Wins

ImageNet has 1,000 categories — only a handful involve human faces (e.g., ‘face powder’, not identity recognition).

FaceNet was trained on 200 million face images across millions of identities — it has seen every type of lighting, angle, and expression.

FaceNet's 512-dimensional embeddings encode identity information so effectively that even a simple k-NN classifier achieves 92%+ accuracy.

Key insight: the domain of pretraining matters as much as the scale. Face-specific knowledge > general visual knowledge for face tasks.

This is Section 1.4 Domain Adaptation in action: a model pretrained in the same visual domain (faces) needs far less adaptation.

System Architecture — The Complete Pipeline

The face recognition system is built as a two-stage pipeline. Stage 1 detects and crops faces from images. Stage 2 recognises whose face it is. This separation of concerns is a best practice in production face recognition systems.

🔍 Full Pipeline — Step by Step

INPUT: Raw photograph (any size, one or more people)

STAGE 1 — FACE DETECTION (MTCNN)

Step 1: MTCNN P-Net scans image at multiple scales — finds candidate face regions

Step 2: MTCNN R-Net refines candidates — rejects false positives

Step 3: MTCNN O-Net finds 5 facial landmarks (eyes, nose, mouth corners)

Step 4: Align and crop face to 160×160 pixels using landmark positions

Output: Clean, aligned face crop(s)

STAGE 2 — FACE RECOGNITION (FaceNet)

Step 5: FaceNet encoder processes the 160×160 face crop

Step 6: Produces a 512-dimensional embedding vector (the face's ‘fingerprint’)

Step 7: SVM classifier compares embedding to stored embeddings of 5 known people

Output: Predicted identity + confidence score

Component	Model	Role	Trainable?	Why This Choice
Face Detector	MTCNN	Find and align faces in any photo	No — frozen	Extremely robust detector; handles occlusion, angle, lighting variation

Component	Model	Role	Trainable?	Why This Choice
Face Encoder	FaceNet (Inception-ResNet-v1)	Convert face image to 512-d identity vector	No — frozen	Pretrained on 200M faces; embeddings encode identity, not appearance
Classifier	SVM (RBF kernel) or k-NN	Map embedding to one of 5 identities	Yes — you train this	With 100 samples, SVM/k-NN outperform neural classifiers; no overfitting

Why SVM Instead of a Neural Network Classifier?

With only 20 embeddings per person (100 total), a neural network classifier would overfit immediately. SVM with RBF kernel is mathematically designed to find the optimal decision boundary even with very few samples.

k-NN (k=1 or k=3) also works well: it simply finds the nearest stored embedding and predicts that person's identity.

Both SVM and k-NN require no gradient descent — they train in milliseconds on 100 samples.

At larger scales (1,000+ images per person), a small MLP classifier would become more competitive.

Phase 0 — Data Collection (Before the Lab)

You need 20 images of each of 5 people — 100 images total. Quality of data collection directly determines recognition accuracy. Follow these guidelines to maximise variety within your small dataset:

Variation Type	What to Capture	Why It Matters	Target Count
Lighting	Indoor overhead, near window, outdoor shade, outdoor sun	FaceNet is robust to lighting but your classifier must see the variation	5 images
Angle	Front-facing, slight left turn, slight right turn, slight tilt up/down	Real-world recognition must handle head pose variation	5 images

Variation Type	What to Capture	Why It Matters	Target Count
Expression	Neutral, smiling, serious, surprised, eyes squinted	Expression changes geometry; classifier must see multiple examples	5 images
Background	Plain wall, cluttered room, outdoors	Background should not leak into the face crop after MTCNN alignment	3 images
Distance	Close-up (face fills frame), medium (head+shoulders), slightly far	Tests MTCNN's detection at different resolutions	2 images

⚠ Common Data Collection Mistakes to Avoid

All 20 images taken on the same day in the same pose: the classifier memorises that exact appearance, not the person.

Images where MTCNN cannot detect a face (too dark, extreme angle, sunglasses): these become empty training examples.

Multiple faces in the frame without cropping: FaceNet will embed the wrong face.

File naming inconsistency: organise as /data/Person_Name/image_001.jpg through image_020.jpg before starting.

JPEG compression artefacts from heavily compressed phone photos: use highest quality camera settings.

Recommended folder structure:

📁 Dataset Folder Structure

```

face_recognition_project/
├── data/
│   ├── raw/                ← original photos before processing
│   │   ├── Person_1/       ← use real first names (e.g., Alice/)
│   │   │   ├── image_001.jpg
│   │   │   ├── image_002.jpg
│   │   │   └── ... (20 images)
│   │   ├── Person_2/
│   │   ├── Person_3/
│   │   ├── Person_4/
│   │   └── Person_5/
│   ├── processed/          ← MTCNN-cropped, aligned face chips (160×160)
│   │   ├── Person_1/
│   │   └── ...
│   ├── embeddings/         ← saved FaceNet embedding vectors (.npy files)
│   ├── models/             ← saved SVM classifier (.pkl file)
│   └── results/            ← confusion matrices, accuracy plots

```

Phase 1 — Face Detection and Alignment (MTCNN)

In this phase, you will run MTCNN over all 100 raw images, detect the face in each image, align it using the detected facial landmarks, and save the cropped 160×160 face chip. This preprocessing step is critical — FaceNet was trained on aligned faces and performs poorly on misaligned inputs.

What MTCNN Does to Each Image

P-Net (Proposal Network): Sweeps a small 12×12 CNN across the image at multiple scales. Proposes ~1,000 candidate face regions.

R-Net (Refinement Network): Takes all candidate crops, resizes to 24×24, rejects non-faces. Reduces candidates to ~50.

O-Net (Output Network): Takes remaining candidates at 48×48. Outputs final bounding box + 5 landmark points.

Alignment: Uses the eye positions to compute an affine transformation. Rotates and crops the face so eyes are always horizontal.

Result: A standardised 160×160 face image where all faces are in consistent, canonical pose regardless of original photo angle.

Expected outputs from Phase 1:

- 100 face crops saved to data/processed/, one subfolder per person
- Detection log showing which images succeeded and which failed (flag failures for replacement)
- Visual check: display a 5×4 grid of cropped faces for each person to confirm alignment quality
- Expected success rate: 95–100% if photos were taken following the data collection guidelines above

If MTCNN Fails to Detect a Face

Lower the detection confidence threshold (default 0.9 → try 0.7) — useful for slightly blurry or dark images.

Check if the face is smaller than 40×40 pixels in the original image — too small for reliable detection.

Check for extreme head angles (profile view, looking up/down more than 30°).

If MTCNN fails on more than 3 images for one person, retake those photos before proceeding.

A missing training image is worse than a slightly imperfect one — aim for all 20 per person.

Phase 2 — Generating FaceNet Embeddings

FaceNet encodes each 160×160 face image into a 512-dimensional vector — the face’s numerical fingerprint. FaceNet was trained using a triplet loss: it learned to make embeddings of the same person close together and embeddings of different people far apart, regardless of lighting or angle. You will run all 100 face chips through the frozen FaceNet encoder and save the resulting embeddings.

💡 Understanding FaceNet Embeddings

Each face becomes a point in 512-dimensional space. The embedding captures identity, not appearance. Two photos of the same person in different lighting: embeddings are close together (cosine similarity > 0.9).

Two different people with similar appearance: embeddings are far apart (cosine similarity < 0.5).

FaceNet was trained with triplet loss: for any anchor face, its embedding is closer to the same person than any other person.

L2-normalisation is applied to all embeddings — they lie on the surface of a unit hypersphere in 512-D space.

This means Euclidean distance and cosine similarity give equivalent results after normalisation.

What you will observe in Phase 2:

- Each person's 20 embeddings cluster tightly together in 512-D space — visualise with t-SNE or PCA reduced to 2D
- Different people's clusters are clearly separated — even before training any classifier
- Intra-person distance (same person, different photos) is much smaller than inter-person distance
- Save all 100 embeddings as a (100, 512) numpy array with corresponding labels

🔍 t-SNE Visualisation — What to Expect

Run t-SNE on your 100 embeddings (100 points in 512-D reduced to 2-D for plotting).

You should see 5 distinct clusters, one per person, with minimal overlap.

Tight clusters: FaceNet embeddings are highly consistent for the same person.

If two people's clusters overlap significantly, they may look very similar — the classifier will struggle on those two.

This visualisation directly shows whether your 20 images per person adequately represent their appearance variation.

A well-separated t-SNE plot almost guarantees high classification accuracy before you even train the classifier.

Phase 3 — Training the Classifier

With 100 embeddings (20 per person) saved, you will now train a classifier. You will train and compare two classifiers — SVM and k-NN — to observe how each handles the small dataset. An 80/20 train-test split gives 80 training examples and 20 test examples (4 per person in the test set).

Classifier	How It Works on Embeddings	Hyperparameters to Tune	Expected Accuracy
SVM (RBF kernel)	Finds the maximum-margin hyperplane separating each person's embeddings from all others. One-vs-rest for 5 classes.	C (regularisation strength): try 0.1, 1.0, 10.0. Gamma: try 'scale' and 'auto'	92–97%

Classifier	How It Works on Embeddings	Hyperparameters to Tune	Expected Accuracy
k-NN (k=1)	For each test embedding, find the single nearest training embedding. Predict that person's label.	k: try 1, 3, 5. Distance metric: euclidean, cosine	88–95%
k-NN (k=5)	Vote among 5 nearest neighbours. More robust than k=1 but needs enough samples per class.	Same as k-NN, but k=5 requires at least 5 training samples per class	85–93%
Linear SVM	Simpler than RBF — straight hyperplane in 512-D space. Often surprisingly effective on L2-normalised embeddings.	C (regularisation): try 0.01, 0.1, 1.0, 10.0	90–95%

Cross-Validation Strategy for 100 Samples

With only 100 samples, a single 80/20 split is fragile — the test set is only 20 images.

Use 5-fold cross-validation: split 100 samples into 5 folds of 20 each. Train on 80, test on 20, repeat 5 times, average results.

This gives a much more reliable accuracy estimate than a single split.

Stratified folds: ensure each fold has exactly 4 images per person (not random — could otherwise exclude a person from test set).

Report both mean accuracy and standard deviation across folds: e.g., 94.2% ± 2.1%.

The standard deviation tells you how stable the model is — high std means performance varies with which images end up in the test set.

Phase 4 — Evaluation and Analysis

Accuracy alone is not enough — you need to understand where and why the model makes mistakes. This phase covers three analysis steps: confusion matrix analysis, error case inspection, and threshold-based rejection.

4.1 Confusion Matrix Analysis

A confusion matrix shows which people get confused with which. With 5 people, it is a 5×5 grid where the diagonal shows correct predictions and off-diagonal cells show errors. This is far more informative than a single accuracy number.

- Diagonal cells (correct): should be high for all 5 people — unequal performance across people reveals data quality issues
- Off-diagonal cells (confusions): if Person 2 is often confused with Person 4, their embeddings overlap — check if they look similar or if one has poor image variety
- Asymmetric confusions: if Person 2 is predicted as Person 4 more than Person 4 is predicted as Person 2, check for data imbalance in those two classes

4.2 Error Case Inspection

For every misclassified test image, display: (1) the original photo, (2) the MTCNN-cropped face, (3) the predicted person with confidence, (4) the correct person. This visual inspection almost always reveals the root cause:

- Alignment failure: MTCNN misaligned the face — the crop shows part of the background or a tilted face
- Extreme lighting: one very dark or overexposed image that doesn't appear in the training set
- Genuine similarity: two people who look similar — the model is making an understandable mistake
- Bad training image: the one training image for this condition was poorly lit — the test image has no near neighbour

4.3 Confidence Threshold and Rejection

Real production face recognition systems never output a prediction with zero confidence — they include a rejection threshold. If the model is not confident enough, it says “Unknown Person” rather than guessing.

💡 Implementing Confidence Thresholding

For SVM: use `predict_proba()` to get class probabilities. If max probability < 0.70 , predict ‘Unknown’.
For k-NN: if the nearest neighbour distance $>$ threshold (e.g., 0.8 in cosine distance), predict ‘Unknown’.
Test the threshold by presenting a photo of a 6th person not in the training set — the model should output ‘Unknown’.

Production systems like Face ID use very tight thresholds (False Acceptance Rate $< 0.001\%$) to prevent false recognition.

The tradeoff: tighter threshold = fewer false accepts, but more legitimate users rejected. Tune based on your security requirement.

Ablation Study — Measuring Every Design Choice

Run the following experiments in order. Each adds one component and measures its independent contribution. This directly demonstrates the Day 9 concepts of transfer learning, domain-specific pretraining, and feature extraction.

Experiment	What Changes	Expected Accuracy	What This Teaches
Baseline: k-NN on raw pixels	No model at all — compare flattened pixel vectors directly	25–40%	Raw pixels have no semantic structure; appearance variation destroys recognition
+ ResNet-50 (ImageNet) Features	Use frozen ResNet-50 as feature extractor; SVM classifier on top	70–82%	General visual features help significantly but are not face-specific

Experiment	What Changes	Expected Accuracy	What This Teaches
+ FaceNet Features	Replace ResNet-50 with frozen FaceNet; same SVM classifier	92–97%	Domain-specific pretraining provides dramatically better features for this task
+ Face Alignment (MTCNN)	Add MTCNN preprocessing before FaceNet (vs. no alignment)	+2–4% gain	Alignment standardises face geometry — reduces intra-person embedding variance
+ Confidence Thresholding	Reject predictions below 70% confidence	Precision ↑ by ~3%	Rejecting uncertain predictions improves precision at the cost of recall
Full Pipeline (best)	MTCNN + FaceNet + SVM + Threshold	93–97%	All components work together; each contributes a measurable improvement

Expected Results and Benchmarks

Metric	Expected Value	What Good Looks Like	Red Flag
Overall Accuracy (5-fold CV)	92–97%	All 5 people correctly identified >90% of the time	<80% — likely a data collection or alignment problem
Per-person Accuracy	>88% for all 5	No single person dramatically worse than others	One person <70% — check that person’s image variety
Intra-person Embedding Std	<0.15 (L2-normalised)	Tight cluster in t-SNE — all 20 images close together	>0.3 — large appearance variation; some images may be misaligned
Inter-person Min Distance	>0.5 (cosine distance)	Well-separated clusters in t-SNE	<0.3 — two people look very similar; may need more varied photos
Unknown Person Rejection	>95% rejection rate	6th unknown person triggers ‘Unknown’ output consistently	<80% — threshold too loose; tighten confidence threshold
MTCNN Detection Rate	>95% of photos	Almost all photos yield a valid face crop	<85% — photos were taken in poor conditions; retake failed images

Real-World Connection: How This Scales

This exact pipeline (MTCNN + FaceNet/ArcFace + classifier) is used in production systems at scale.

Facebook's DeepFace (2014) used a similar architecture and achieved 97.35% on LFW benchmark — matching human performance.

Apple's Face ID uses an infrared version of this approach with a 3D depth sensor — the math is the same, the sensing modality differs.

Chinese surveillance systems use ArcFace embeddings with SVM/cosine classifiers across millions of registered identities.

The main difference between your 5-person system and a production system: scale (millions of embeddings), hardware (dedicated NPU), and security (anti-spoofing checks).

Your 100-image proof of concept demonstrates the exact same architectural principles.

Debugging Guide

Problem	Likely Cause	Fix
MTCNN detects 0 faces in many images	Images too dark, face too small, extreme angle, or low resolution	Increase brightness, move closer, adjust angle. Lower MTCNN threshold from 0.9 to 0.7
Accuracy below 80% with FaceNet	Poor image variety — all 20 images of one person look identical	Retake photos with different lighting, angles, and expressions as specified in Phase 0
One person has much lower accuracy	Their photos lack variation or are consistently poorly lit	Check that person's crops visually. Retake 5+ images with different lighting
t-SNE shows overlapping clusters for 2 people	Those two people genuinely look similar to FaceNet, or misalignment is high	Add more varied photos; check alignment quality for both people
SVM predictions all same class	Class imbalance after MTCNN failures reduced one person's sample count significantly	Ensure all 5 people have equal sample counts. Retake missing images
Unknown person gets recognised as Person X	Confidence threshold too low; that person looks similar to Person X	Tighten threshold (e.g., 0.70 → 0.80). Check the specific embedding distance

Lab Submission Checklist

- Phase 0: 100 images collected (20 per person) following the variety guidelines
- Phase 1: All images processed by MTCNN. Detection rate >95%. Sample grid of crops reviewed.
- Phase 2: 100 embeddings generated and saved. t-SNE plot shows 5 separated clusters.
- Phase 3: SVM and k-NN both trained and evaluated. 5-fold CV accuracy reported with standard deviation.
- Phase 4: Confusion matrix plotted. Error cases inspected. Unknown person rejection tested.
- Ablation study: All 6 rows completed. Accuracy gain from FaceNet vs ResNet-50 documented.

- Reflection (written, 150 words): Explain in your own words why FaceNet outperformed ImageNet-pretrained ResNet-50 for this task, connecting your result to Section 1.4 (Domain Adaptation).

10 — Glossary & Review Questions

10.1 Key Terms

Term	Definition
Transfer Learning	Reusing a model trained on one task as the starting point for a different but related task.
Feature Extraction	Using a frozen pretrained backbone as a fixed feature extractor; only the new head is trained.
Fine-Tuning	Continuing to train some or all layers of a pretrained model on a new dataset with a small learning rate.
Catastrophic Forgetting	The phenomenon where training on a new task causes a model to forget knowledge from its previous training.
Domain Adaptation	Techniques for adapting a model trained on one data distribution to a different but related distribution.
Covariate Shift	A change in the distribution of inputs between training and deployment, while the task remains the same.
Backbone	The main feature-extraction network in a model; typically a pretrained CNN or Transformer.
ImageNet	A dataset of 1.2 million images across 1,000 classes; the primary source of pretrained weights for vision.
ResNet	A deep CNN architecture using skip connections to solve the vanishing gradient problem.
EfficientNet	A CNN family using compound scaling of depth, width, and resolution simultaneously.
MobileNet	A lightweight CNN architecture using depthwise separable convolutions for mobile/edge deployment.
Vision Transformer (ViT)	A Transformer applied directly to sequences of image patches; no convolutions used.
Patch Embedding	The projection of a flattened image patch to a fixed-dimensional token for ViT processing.
Positional Encoding	A signal added to token embeddings to inform the Transformer of each token's position in the sequence.
Progressive Unfreezing	Unfreezing one layer group at a time, starting from the deepest, to prevent catastrophic forgetting.
Discriminative LR	Using different learning rates for different layer groups; smaller for early layers, larger for later layers.
Mixup	An augmentation technique that blends two images and their labels proportionally using a mixing coefficient.
CutMix	An augmentation technique that pastes a rectangular patch from one image into another; labels mixed by area.

Term	Definition
RandAugment	A simplified augmentation policy: randomly select N operations and apply at magnitude M.
AutoAugment	A learned augmentation policy discovered by reinforcement learning; expensive but effective.
TTA (Test-Time Augmentation)	Running inference on multiple augmented versions of an image and averaging the predictions.
Self-Attention	A mechanism allowing each token in a sequence to attend to all other tokens using Q, K, V projections.
Query (Q)	A projection of a token representing what the token is searching for in the sequence.
Key (K)	A projection of a token representing what information it contains for other tokens to attend to.
Value (V)	A projection of a token representing the information it contributes when attended to.
Scaled Dot-Product Attention	The core attention operation: $\text{softmax}(QK^T / \sqrt{d_k}) \cdot V$.
Multi-Head Attention	Running multiple attention heads in parallel, each learning different relationship types.
BERT	Bidirectional Encoder Representations from Transformers; a bidirectional encoder pretrained with masked LM.
GPT	Generative Pretrained Transformer; a causal decoder pretrained by predicting the next token.
Gradient Clipping	Scaling gradients to prevent exploding updates when the gradient norm exceeds a threshold.
Label Smoothing	Softening hard one-hot labels to prevent overconfident predictions and improve calibration.
Mixed Precision	Using FP16 for computation and FP32 for weight storage; halves memory and doubles speed.
Gradient Accumulation	Accumulating gradients across multiple small batches before updating weights; simulates large batches.
EMA (Exponential Moving Average)	A smoothed running average of model weights that generalises better than final training weights.
Pruning	Removing weights or entire neurons/filters with small magnitudes to compress a neural network.
Quantization	Reducing numerical precision (e.g., FP32 $\rightarrow$ INT8) to reduce model size and increase inference speed.
PTQ (Post-Training Quantization)	Applying quantization to an already-trained model without retraining; fast but can lose accuracy.
QAT (Quantization-Aware Training)	Training with simulated quantization so the model learns to be robust to precision reduction.

Term	Definition
Knowledge Distillation	Training a small student model to mimic the soft output distributions of a large teacher model.
Temperature (T)	A parameter that controls how soft the teacher's output distribution is in knowledge distillation.
ONNX	Open Neural Network Exchange; a framework-agnostic format for deploying trained models.
GAN	Generative Adversarial Network; two networks (Generator, Discriminator) trained adversarially.
Mode Collapse	A GAN training failure where the Generator produces only a small subset of possible outputs.
VAE	Variational Autoencoder; encodes inputs as distributions (μ , σ) to create a smooth, continuous latent space.
ELBO	Evidence Lower Bound; the VAE training objective combining reconstruction loss and KL divergence.
Diffusion Model	A generative model that learns to reverse a gradual noising process to generate data from noise.
DDPM	Denoising Diffusion Probabilistic Model; the foundational diffusion model architecture (Ho et al., 2020).

10.2 Conceptual Questions

Q1. Explain in your own words why a neural network pretrained on ImageNet (natural photos of cats, dogs, and cars) can help a model learn to detect tumours in X-ray scans. What specific knowledge transfers, and what must be relearned?

Q2. A company has 500 labelled satellite images of flooded regions. They want to train a model to detect flooding in new images. Walk through the decision framework from Section 1.3 and justify which fine-tuning strategy you would choose.

Q3. Explain catastrophic forgetting using a real-world analogy from human learning. Then explain how progressive unfreezing addresses this problem in neural networks.

Q4. Why does the Transformer need positional encoding when a recurrent neural network (RNN) does not? Describe what would happen to a Transformer if positional encodings were removed.

Q5. Compare BERT and GPT: if you were building (a) a legal contract analysis system to classify contract types, and (b) a chatbot to answer customer support questions, which architecture would you choose for each? Justify your answer.

10.3 Practical Questions

Q6. You are fine-tuning EfficientNet-B4 on a medical imaging dataset. During Phase 2 (full fine-tuning), the validation loss suddenly spikes after epoch 3 even though training loss continues to decrease. List three possible causes and the fix for each.

Q7. Explain why Mixup and CutMix both improve model accuracy and calibration, but through different mechanisms. Give a specific example of a task where CutMix would be more beneficial than Mixup.

Q8. A researcher with a single 8GB GPU wants to achieve the same effective batch size of 256 as a paper that trained with 32 GPUs. The maximum batch size that fits in 8GB is 8 samples. Explain how gradient accumulation achieves this and whether the final model will be equivalent.

Q9. Your company needs to deploy a PyTorch model on both NVIDIA GPUs (for the server API) and iPhones (for the mobile app). Describe the deployment pipeline you would use, naming the specific format and runtime for each target.

Q10. Compare the training stability of GANs vs diffusion models. Why did diffusion models rapidly displace GANs for high-quality image generation despite being much slower at inference?

10.4 Challenge Questions

Q11. In self-attention, the scaling factor $1/\sqrt{d_k}$ is applied to the dot products before softmax. Explain mathematically and intuitively why this scaling is necessary. What would happen to training if it were removed?

Q12. Knowledge distillation uses temperature T to soften the teacher's output distribution. Explain why a temperature of $T=1$ provides less useful information to the student than $T=4$. Use a concrete example with actual probability values to illustrate the difference.

Q13. The VAE's ELBO loss has two components: reconstruction loss and KL divergence. Explain what happens to the latent space if you set the KL weight to zero (removing the regularisation term). Why does this make the model a worse generative model even if reconstruction quality improves?

The future of AI is bright, and you can be part of it!